DS605 · MACHINE LEARNING

Ensemble Methods

Lecture notes for Lectures 29–31: voting, bagging, Random Forests, boosting and stacking, taught through ShopKart's cash-on-delivery refusals

Arpit Rana

Lecture 29 · Tuesday 27th October 2026 Lecture 30 · Thursday 29th October 2026 Lecture 31 · Friday 30th October 2026

About these notes

These notes follow the order of the slides for Lectures 29–31 and fill in the reasoning that the slides show only briefly. Each "Think first" box contains a question asked in class, followed by its answer. Equations are numbered by lecture, so (29.6) is the sixth numbered equation of Lecture 29. Every number in these notes was computed with scikit-learn 1.9.1 on the ShopKart data described below, so the results can be reproduced exactly.

The ShopKart problems used throughout

Main problem: will a COD parcel be refused?

When a customer refuses a cash-on-delivery parcel at the door, ShopKart pays shipping both ways and the item is out of stock for about a week. We want to predict a refusal **before dispatch**. The training set has 1,000 past COD orders, of which 42.6% were refused; a further 5,000 new orders are used for testing. Table 0.1 lists the nine columns.

Column	Meaning	Example
delivery days	days from order to doorstep	1.9
order value	basket value in ₹ thousand	7.8
past refusals	COD parcels this customer refused before	1
tenure	months since the customer joined	26
distance	warehouse to doorstep, in km	244
PIN refusal rate	share of COD parcels refused in the customer's PIN code	0.06
order hour	hour of the day the order was placed (0–23)	18
phone verified	whether the customer's phone number is verified	yes
order-ID digit	last digit of the order ID: **pure noise**, included on purpose	1

Table 0.1. Columns of the COD-refusal table. The target is *refused* (1) or *accepted* (0).

Pilot: two columns for pictures

To draw decision boundaries we use 300 COD orders from one city with only two columns, *days to deliver* and *order value*, and test on 2,000 new pilot orders. In the figures, **blue squares** are accepted orders and **red circles** are refused orders.

Ten orders, A to J

Bootstrap sampling, out-of-bag evaluation and AdaBoost are worked by hand on ten pilot orders: A, B, C, F and I were accepted; D, E, G, H and J were refused.

Delivery time (from Lecture 28)

Gradient boosting is shown on the 200 delivery orders of Lecture 28: distance in km as the input and delivery time in hours as the target.

Notation

Symbol	Meaning
N	number of members (predictors) in the ensemble
$h_j(x)$	prediction of member j for order x
$\hat{y}(x)$	prediction of the ensemble
p	accuracy of one voter
n	number of voters; m is the number of training orders
c	probability that a voter copies a shared answer (Section 29.7)
σ^2	variance of one tree's prediction
ρ	correlation between the predictions of two trees
$\bar{h}$	average prediction of the trees
$w^{(i)}$	weight of training order i in AdaBoost
r_j, α_j	weighted error rate and predictor weight of AdaBoost member j
η	learning rate
$r_i = y_i - F(x_i)$	residual of order i in gradient boosting

Lecture 29 · Many Voices: Voting and Bagging

Tuesday 27th October 2026. This lecture asks why a group of models can be better than its best member. We answer with voting classifiers, then build a whole crowd from one algorithm with bagging and pasting, and finish with out-of-bag evaluation.

29.1 Three models: which one do we deploy?

We trained three different models on the 1,000 past COD orders and tested each on 5,000 new orders.

Model	Accuracy on 5,000 new orders
Logistic Regression	77.1%
Random Forest	80.3%
SVM (RBF kernel)	80.2%

THINK FIRST

The three accuracies are close. Which model should ShopKart put into production?

Answer. Perhaps none of them alone. The models are close in accuracy, but they do not make the **same** mistakes: on 21% of the new orders at least one model disagrees with the other two. If we let the three models **vote**, the combined model scores **80.8%**, which is better than every member. This combined model is an **ensemble**.

29.2 The wisdom of the crowd

Suppose 300 warehouse staff each guess the weight of a parcel that truly weighs 4.20 kg. One person is typically wrong by about 0.6 kg. The **average** of all 300 guesses is 4.23 kg: almost exact. Individual errors point in different directions, so in the average they **cancel out**. Francis Galton observed the same effect in 1906, when 787 visitors at a country fair guessed the weight of an ox and the middle guess was within 1% of the truth.

The effect needs one condition: the guesses must be **independent**. If many people copy one loud colleague, they share his error, and averaging cannot remove an error that everyone shares. Figure 29.1(b) shows the error of the crowd average levelling off at a floor when people copy.

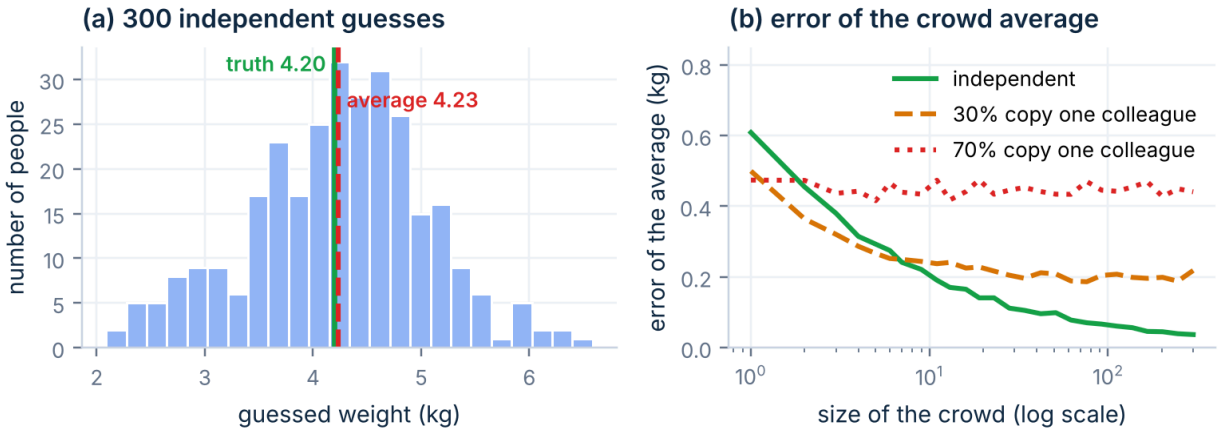


Figure 29.1. (a) 300 independent guesses of a parcel weight; the average (red) lies close to the truth (green). (b) The error of the average falls as the crowd grows, but only to a floor when part of the crowd copies one colleague.

29.3 What is an ensemble?

> **Definition**
> A group of predictors (classifiers or regressors) is called an **ensemble**. The technique is called **ensemble learning**, and an algorithm that does it is called an **ensemble method**.

A familiar example is the **Random Forest**: train many decision trees, each on a different random subset of the orders; to predict, ask every tree and return the class with the most votes. It is simple, yet it is one of the most powerful methods for tabular data.

Three terms recur in these notes:

- **Base learner** (or **member**): one model inside the ensemble.
- **Weak learner**: a model that is only slightly better than random guessing.
- **Strong learner**: a model with high accuracy.

The central claim of these lectures is that many weak learners, combined well, can form a strong learner.

29.4 Three ways to build an ensemble

Every method in Lectures 29–31 answers the same question in a different way: *how do we make the members different enough that their errors cancel?*

Idea	How the members differ	Methods
1. Different algorithms	Same data; different kinds of model	Voting (L29), Stacking (L31)
2. Different data	Same algorithm; each copy trained on a different random sample, in parallel	Bagging, Pasting (L29); Random Forest, Extra-Trees (L30)
3. Learn from mistakes	Same algorithm; copies trained one after another, each correcting the previous ones	AdaBoost (L30); Gradient Boosting (L31)

29.5 Voting classifiers

The ShopKart problem

When a customer refuses a **cash-on-delivery (COD)** parcel at the door, ShopKart pays shipping both ways and the item is out of stock for about a week. If we can predict a refusal **before dispatch**, we can call the customer to confirm or ask for a small prepayment. Our main table has 1,000 past COD orders, of which 42.6% were refused, and nine columns (Table 0.1). The last column, the last digit of the order ID, is pure noise and has been included on purpose.

To draw pictures we also use a **pilot** of 300 orders from one city with only two columns: *days to deliver* (x_1) and *order value* in ₹ thousand (x_2). Customers lose patience with slow deliveries, and cheap impulse buys and expensive items lose it fastest, so the boundary between accepted and refused orders is curved.

Diverse predictors

Figure 29.2 shows four very different models trained on the same 300 pilot orders. Each draws a different boundary. Each is right on roughly 80–90% of new orders, but they are wrong on **different** orders. That difference is what a vote can exploit.

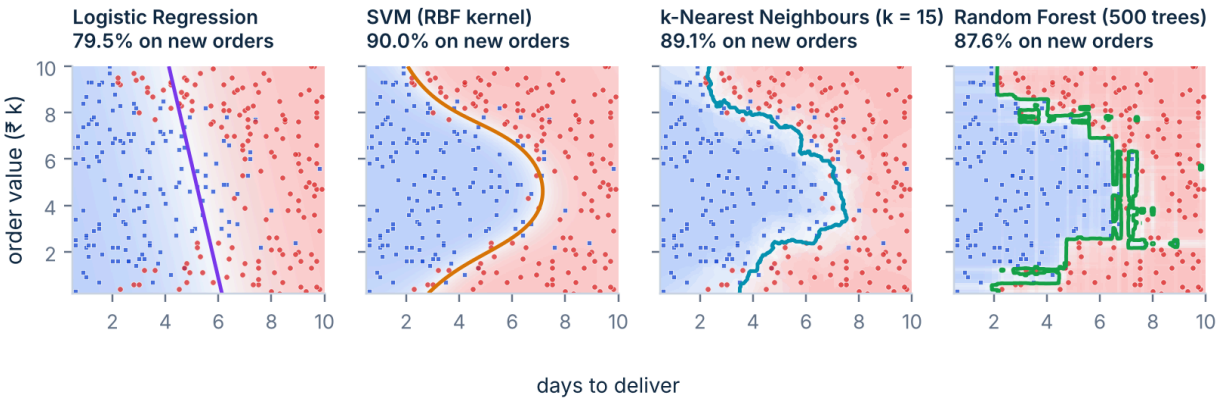


Figure 29.2. Four different models on the same 300 pilot orders. Shading shows the predicted probability of refusal; the line is the 0.5 boundary. Squares: accepted orders; circles: refused orders.

Hard voting

A **hard voting classifier** asks each member for its predicted class and returns the class that gets the most votes, the statistical **mode**:

$$\hat{y}(x) = \text{mode}\{\, h_1(x),\, h_2(x),\, \dots,\, h_N(x) \,\} \tag{29.1}$$

Here $h_j(x)$ is the class predicted by member j for order x, and N is the number of members. With an even number of members a tie is possible; scikit-learn then picks the class that comes first in sorted order (here, *accepted*). Using an odd number of voters avoids ties.

29.6 Why does voting work?

THINK FIRST

Imagine 1,000 models. Each is right only 51% of the time, barely better than tossing a coin. If they vote, how often is the majority right: close to 51%, or much higher?

Answer. Much higher: about **73%**. With 10,000 such voters it rises to about **98%**, provided the voters are independent. The reason is the law of large numbers.

A slightly biased coin

Think of each voter as a coin that lands heads (right) with probability 0.51. Figure 29.3 shows ten series of 10,000 tosses. Early on, the proportion of heads jumps around and many series show a minority of heads. As tosses accumulate, every series settles near 0.51, above the 0.5 line. A small edge on each toss becomes a near-certain majority over many tosses.

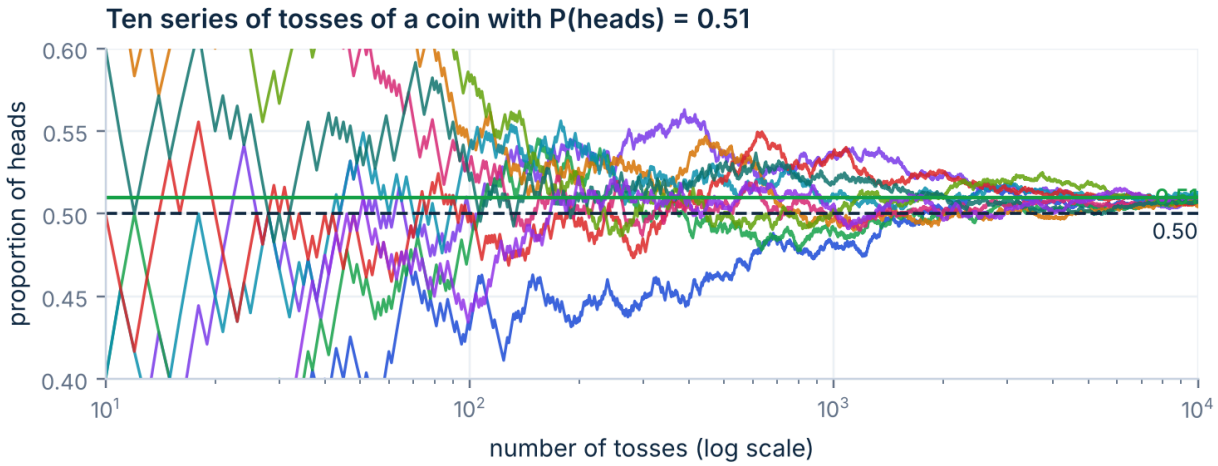


Figure 29.3. Running proportion of heads for ten series of tosses of a coin with P(heads) = 0.51. With more tosses, every series settles above 0.5.

How often is the majority right?

Let n voters (n odd) each be right with probability p, independently. The number of right votes K follows a **binomial distribution**. The majority is right when $K > n/2$, so we add the probabilities of every winning count:

$$P(\text{majority right}) = \Sigma_{k=(n+1)/2}^{n} \; C(n, k) \cdot p^k \cdot (1-p)^{n-k} \quad (29.2)$$

Each term has two parts. $C(n, k)$, read "n choose k", counts the ways to choose *which* k voters are right. The factor $p^k\,(1-p)^{n-k}$ is the probability that those k are right and the other $n-k$ are wrong.

Worked example: three voters, each 70% right

With three voters there are only eight possible outcomes, so we can list them all (R = right, W = wrong).

Outcome	Probability of each	How many	Total	Majority
R R R	$0.7^3 = 0.343$	1	0.343	right
two R, one W	$0.7^2 \times 0.3 = 0.147$	3	0.441	right
one R, two W	$0.7 \times 0.3^2 = 0.063$	3	0.189	wrong

Outcome	Probability of each	How many	Total	Majority
W W W	$0.3^3 = 0.027$	1	0.027	wrong

The majority is right in the first two rows: $0.343 + 0.441 = 0.784$. Each model alone is right 70% of the time; the team is right **78.4%** of the time, and no model was changed.

> **Why the team does better**
> A single model fails whenever it makes a mistake (30%). The team fails only when **at least two models are wrong on the same order**, which is rarer: $0.189 + 0.027 = 0.216$. One model's mistake is usually outvoted by the other two. With five such models the majority is right 83.7% of the time.

From weak learners to a strong learner

Figure 29.4(a) plots equation (29.2) against the number of voters. For $p > 0.5$ the curve climbs towards 100%: a large crowd of weak but independent learners is a strong learner. For $p = 0.45$ the curve falls towards zero: if each voter is **worse than a coin**, the majority amplifies the mistakes and the crowd is worse than one voter.

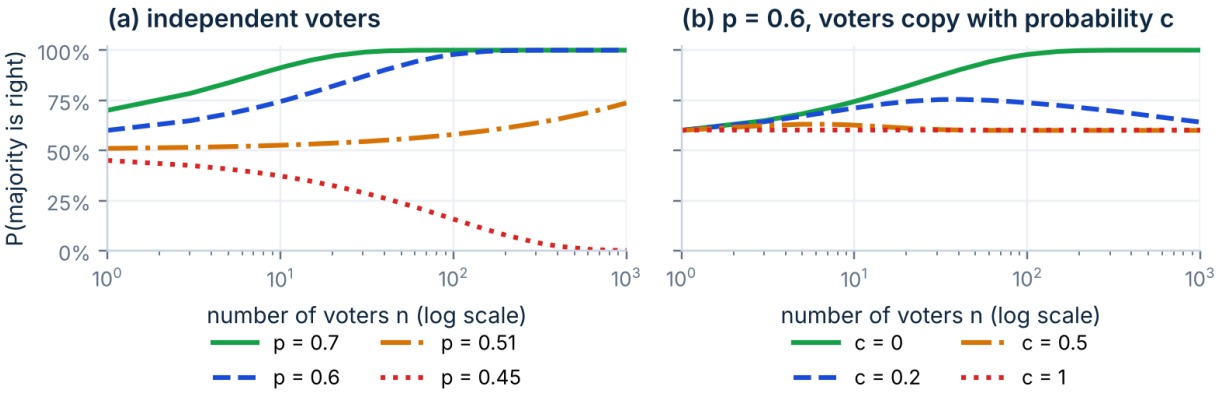


Figure 29.4. (a) Accuracy of a majority vote for independent voters with accuracy p. (b) The same with p = 0.6 when each voter copies a shared answer with probability c.

29.7 When voting fails: voters that copy each other

Equation (29.2) assumes the voters are independent. Real models trained on the same orders are not: they tend to make the **same** mistakes. A simple model of this makes the effect visible.

1. For each order there is one **shared answer**, which is right with probability p. Think of it as the answer the common training data pushes every model towards.
2. Each voter **copies** the shared answer with probability c, or **decides on its own** with probability $1 - c$. A voter that decides on its own is right with probability p.

Each voter on its own is still right with probability $c \cdot p + (1 - c) \cdot p = p$. Copying does not make a voter weaker; it makes voters **wrong together**. (In the slides this slider is labelled ρ; here we call it c to avoid confusion with the correlation ρ of Section 29.10.)

To find the accuracy of the majority, we consider the two possible cases for the shared answer.

- **Case A, the shared answer is right** (probability p). A voter is right if it copies, or if it decides on its own and is right: $q_A = c + (1 - c) \cdot p$.
- **Case B, the shared answer is wrong** (probability $1 - p$). Copying now gives a wrong answer, so a voter is right only if it decides on its own and is right: $q_B = (1 - c) \cdot p$.

Within each case the voters are independent again, so we can reuse the binomial sum of (29.2). Write $\text{Tail}(n, q)$ for that sum with accuracy q. Then

$$P(\text{majority right}) = p \cdot \text{Tail}(n, q_A) + (1 - p) \cdot \text{Tail}(n, q_B) \quad (29.3)$$

With $c = 0$, both cases give $q = p$ and we recover (29.2). With $c = 1$, every voter gives the shared answer and the majority is right with probability p: no gain at all.

Worked example: p = 0.6, c = 0.2, n = 23

- Case A: $q_A = 0.2 + 0.8 \times 0.6 = 0.68$, so $\text{Tail}(23, 0.68) = 0.965$.
- Case B: $q_B = 0.8 \times 0.6 = 0.48$, so $\text{Tail}(23, 0.48) = 0.423$.
- Majority right $= 0.6 \times 0.965 + 0.4 \times 0.423 = 0.748$.

Without copying, the same 23 voters reach 83.6%. A small amount of copying has cost about nine points. Figure 29.4(b) shows something more surprising: with $c = 0.2$, a **larger** crowd eventually does **worse**. When the shared answer is wrong, only 48% of voters remain right, so a large crowd almost always follows the shared answer, and its accuracy falls back towards p.

Conditions for voting to help

1. Better than chance. Each member must be right more often than a random guess.

2. Enough members. The gain grows with the number of voters.

3. Diverse members. Their errors must be as independent as possible. We get diversity by using very different algorithms, or by training on different random subsets of the data.

THINK FIRST

Why is an ensemble of 500 identical copies of the same decision tree useless?

Answer. Identical copies make identical mistakes, so $c = 1$. Every vote is the same, and the majority is just the single tree. Diversity, not size alone, is what helps.

29.8 A voting classifier in scikit-learn

```
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

log_clf = make_pipeline(StandardScaler(), LogisticRegression())
rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
svm_clf = make_pipeline(StandardScaler(), SVC(random_state=42))

voting_clf = VotingClassifier(
    estimators=[("lr", log_clf), ("rf", rnd_clf), ("svc", svm_clf)],
    voting="hard")

for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    print(clf.score(X_test, y_test))
```

Model	Accuracy on new orders
Logistic Regression	77.1%
Random Forest	80.3%
SVM	80.2%
Hard voting	**80.8%**

The vote beats every member, but only by a little. The members are **correlated**: Logistic Regression and the forest agree on 81.5% of new orders, the SVM and the forest on 88.6%, and all three agree on 78.6%. This is the copying effect of Section 29.7 in real data.

29.9 Soft voting

Most classifiers can also give a probability through `predict_proba()`. **Soft voting** averages these probabilities and predicts the class with the highest average:

$$\hat{y}(x) = \arg\max_k \; (1/N) \, \Sigma_{j=1}^{N} \, \hat{p}_j(k \mid x) \tag{29.4}$$

Here $\hat{p}_j(k \mid x)$ is member j's estimated probability that order x belongs to class k. Soft voting gives more weight to **confident** votes.

Worked example: one real order

A new order was delivered in 2 days, worth ₹9,900, from a customer with one past refusal, a PIN-code refusal rate of 0.19, ordered at 9 pm, with an unverified phone. The three models give these probabilities of refusal:

Model	P(refused)	Hard vote
Logistic Regression	0.024	accepted
Random Forest	0.514	refused
SVM (calibrated)	0.731	refused
Average	**0.423**	

Hard voting counts two votes for refused and predicts **refused**. Soft voting averages: $(0.024 + 0.514 + 0.731) / 3 = 0.423 < 0.5$, so it predicts **accepted**. The customer did accept the parcel. The forest was barely above 0.5, while Logistic Regression was very sure of acceptance; soft voting lets that confidence count.

```
from sklearn.calibration import CalibratedClassifierCV

# SVC has no predict_proba by default, so we calibrate it.
# SVC(probability=True) is deprecated in scikit-learn 1.9.
svm_proba = CalibratedClassifierCV(svm_clf, ensemble=False)

voting_clf = VotingClassifier(
    estimators=[("lr", log_clf), ("rf", rnd_clf), ("svc", svm_proba)],
    voting="soft")
voting_clf.fit(X_train, y_train)
voting_clf.score(X_test, y_test)          # 0.810
```

Soft voting scores 81.0% against 80.8% for hard voting. On 140 of the 5,000 new orders the two rules disagree; soft voting is right on 75 of them and hard voting on 65. Every member must provide `predict_proba()`.

29.10 Bagging and pasting

THINK FIRST

Suppose we trust only one algorithm, the decision tree. Trained twice on the same orders, it gives the same tree. How can we still build a diverse crowd of trees?

Answer. Give each tree **different data**: train each tree on a different random sample of the orders. Each tree then learns slightly different rules and makes slightly different mistakes. Lecture 28 showed that trees are unstable, since small changes in the data change the tree. For ensembles, that weakness becomes a strength.

Definitions

Use the **same training algorithm** for every predictor, but train each on a **different random subset** of the training set.

Bagging samples **with replacement** (bootstrap sampling). The name is short for **b**ootstrap **agg**regat**ing** (Breiman, 1996). An order can appear several times in one sample.

Pasting samples **without replacement**. An order appears at most once in a sample, but it can still appear in several samples.

A small example by hand

Take ten past orders, A to J. A bootstrap sample makes ten draws with replacement. Figure 29.5 shows three such samples, the tree trained on each, and their vote on a new order N (6 days, ₹5,000).

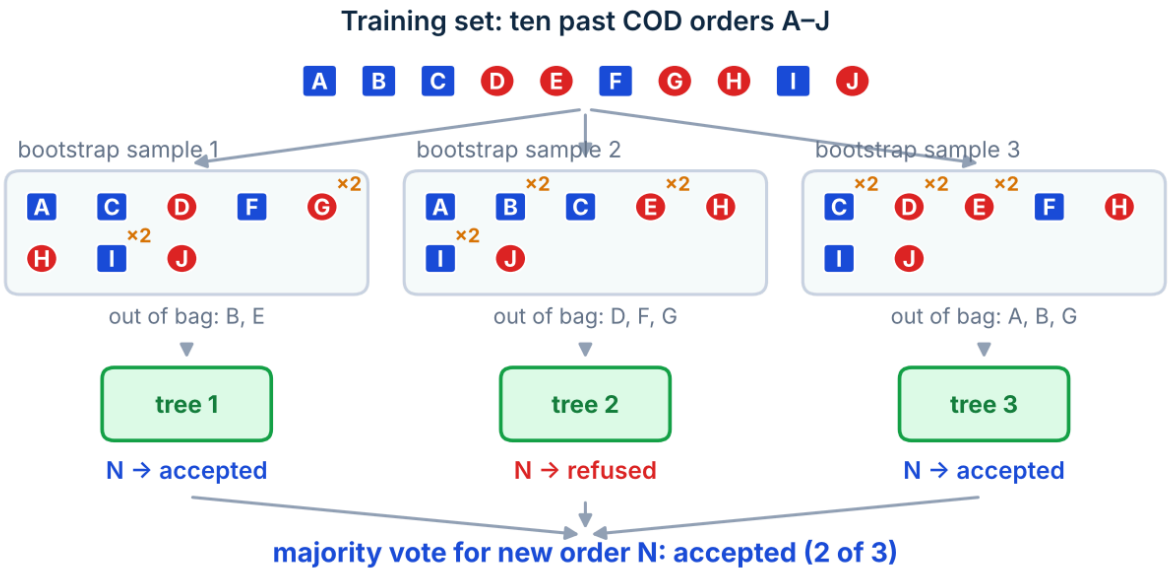


Figure 29.5. Bagging with ten orders. Each bootstrap sample repeats some orders (×2) and leaves others out (out of bag). Each sample trains its own depth-2 tree; the trees vote on the new order N.

Combining the predictors

For classification, bagging returns the **mode** of the members' predictions, as in (29.1), or averages their probabilities when the base model has `predict_proba()`. For regression it returns the **mean**:

$$\hat{y}(x) = (1/N)\ \Sigma_{j=1}^{N}\ h_j(x) \quad (29.5)$$

For example, if three trees predict delivery times of 30, 34 and 35 hours, the ensemble predicts 33 hours.

Each predictor sees fewer distinct orders than the full set, so on its own it has a slightly **higher bias**. Aggregation reduces **variance** strongly. The net result is an ensemble with a **similar bias** but a **lower variance** than one tree trained on all the data.

THINK FIRST

Each bagged tree sees only about 63% of the distinct orders, so each tree is a little worse than one trained on all of them. How can an average of worse trees be better than one good tree?

Answer. A deep tree's main problem is not bias but **variance**: it follows the noise of its particular sample. Different samples give trees with **different** noise. In the average, the shared signal stays and the sample-specific noise partly cancels. We lose a little on bias and gain a lot on variance. The next section measures this.

29.11 Why averaging reduces variance

Fix one new order. Tree j predicts h_j, a random quantity, because tree j was trained on a random sample. The ensemble predicts the average $\bar{h} = (1/N)(h_1 + h_2 + \ldots + h_N)$. Assume that

- every tree has the same variance: $\mathrm{Var}(h_j) = \sigma^2$;
- every pair of different trees has the same correlation ρ, so $\mathrm{Cov}(h_i, h_j) = \rho\sigma^2$ for $i \neq j$ (by the definition $\rho = \mathrm{Cov}(h_i, h_j) / (\sigma \cdot \sigma)$).

Step 1: take out the 1/N. Multiplying a random quantity by a constant a multiplies its variance by a^2, so

$$\mathrm{Var}(\bar{h}) = (1/N^2) \cdot \mathrm{Var}(h_1 + h_2 + \ldots + h_N)$$

Step 2: the variance of a sum. For two terms, $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y)$. In general, the variance of a sum is the sum of **every entry of the covariance table**. For three trees the table is:

	h_1	h_2	h_3
h_1	σ^2	$\rho\sigma^2$	$\rho\sigma^2$
h_2	$\rho\sigma^2$	σ^2	$\rho\sigma^2$
h_3	$\rho\sigma^2$	$\rho\sigma^2$	σ^2

The **diagonal** holds the N variances, giving $N\sigma^2$. The **off-diagonal** holds the covariances between different trees: $N \times N - N = N(N-1)$ cells, giving $N(N-1)\rho\sigma^2$.

Step 3: put the two steps together.

$$\mathrm{Var}(\bar{h}) = (1/N^2) \cdot [\, N\sigma^2 + N(N-1)\rho\sigma^2 \,] = \rho\sigma^2 + ((1-\rho)/N) \cdot \sigma^2 \quad (29.6)$$

(This is equation 15.1 of Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, 2nd edition.)

Reading equation (29.6)

The second term, $(1-\rho)\sigma^2/N$, is each tree's **private** randomness. It shrinks as we add trees.

The first term, $\rho\sigma^2$, is the randomness the trees **share**, because they were trained on overlapping data and pick similar splits. Averaging cannot remove it, however many trees we add. It is a **floor**.

With $\rho = 0$ the variance is σ^2/N. With $\rho = 1$ it stays σ^2, the 500-identical-trees case. For our ShopKart bagged trees we measured $\rho \approx 0.36$, so even infinitely many bagged trees keep about 36% of one tree's variance.

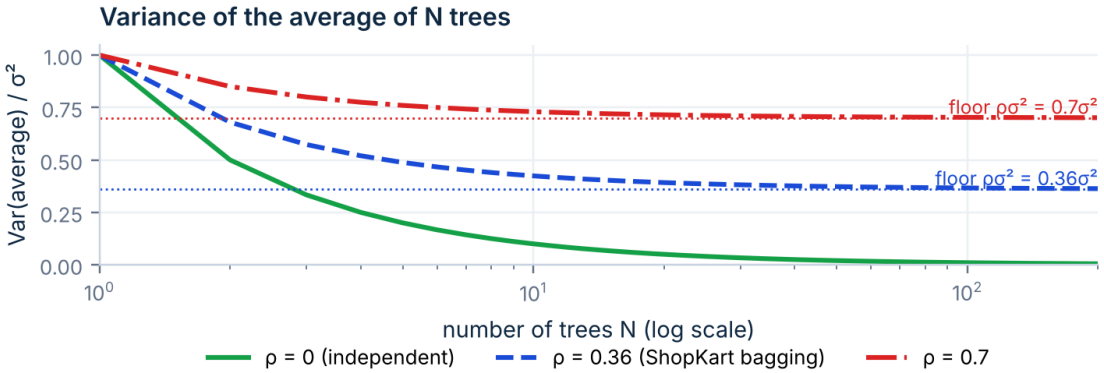


Figure 29.6. Variance of the average of N trees, as a fraction of one tree's variance σ², for three values of the correlation ρ between trees. Dotted lines mark the floors ρσ².

29.12 Bagging in pictures

Figure 29.7 shows eight trees, each trained on its own bootstrap sample of the 300 pilot orders. Each tree carves small islands around noisy orders, but the islands are in **different places**. In the vote, most islands disappear and only regions where most trees agree survive.

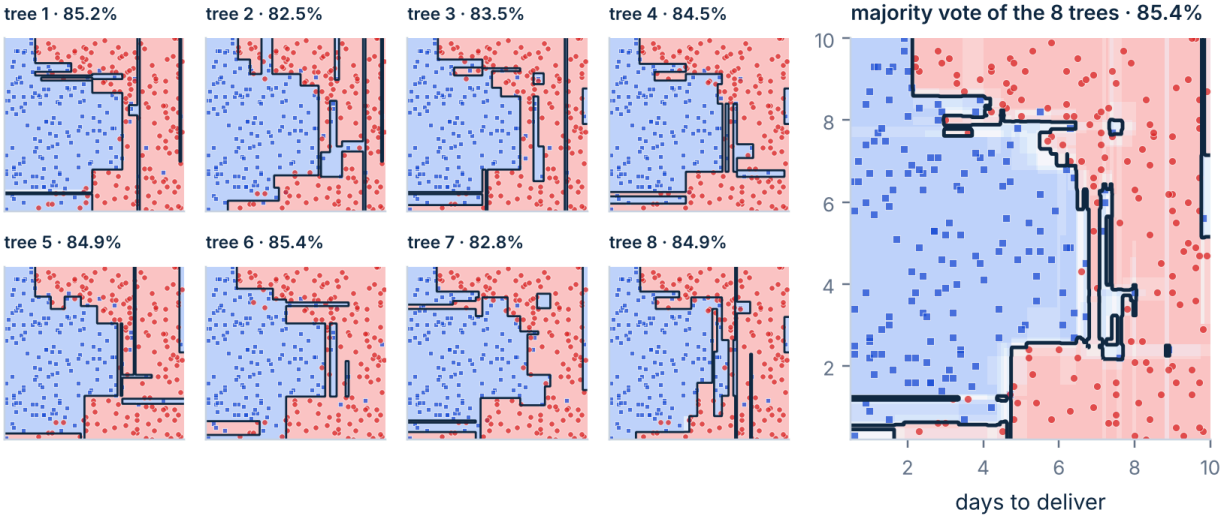


Figure 29.7. Left: eight bootstrap trees and their accuracy on 2,000 new pilot orders. Right: their majority vote.

Figure 29.8 compares one tree with 500 bagged trees. The single tree's boundary is irregular; the ensemble's boundary is smoother, with a similar number of training errors. The ensemble has a comparable bias and a smaller variance. On the pilot, the single unrestricted tree has 40 leaves, fits 99.7% of the training orders and scores 85.2% on new orders; 500 bagged trees score 87.1%.

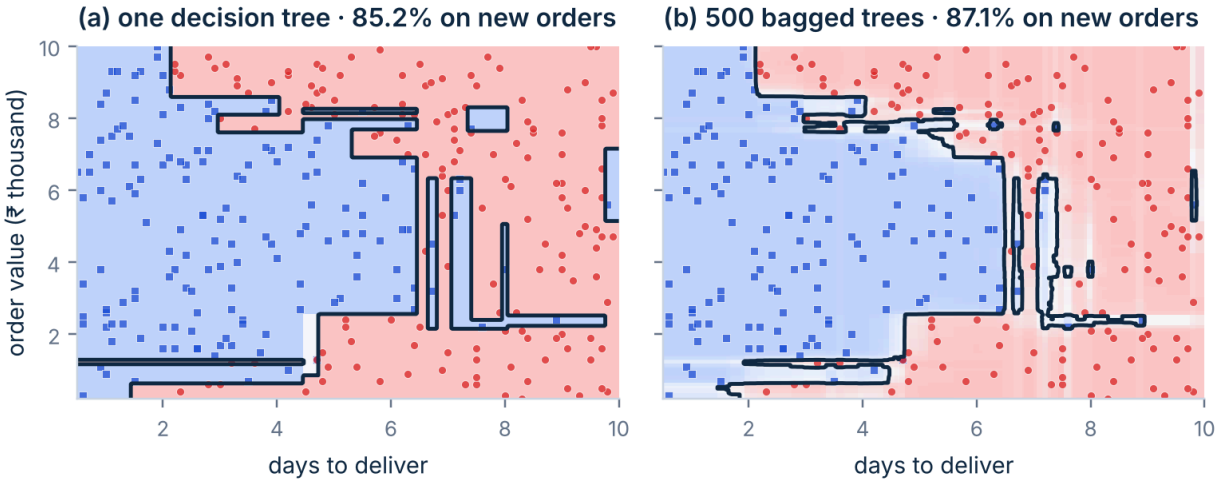


Figure 29.8. A single decision tree (a) and a bagging ensemble of 500 trees (b) on the pilot orders.

29.13 Why do duplicates help in bagging?

Both bagging and pasting show each tree about 63% of the distinct orders. The difference is that bagging also repeats some orders. Figure 29.9(a) shows a typical bootstrap sample of 1,000 orders: 37% of orders are never drawn, 37% are drawn once, 18% twice, 6% three times and 2% four or more times.

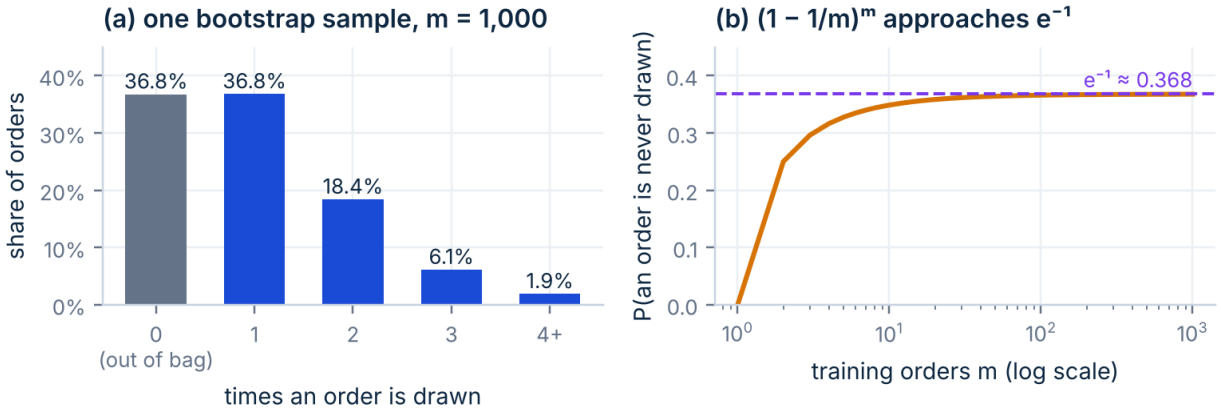


Figure 29.9. (a) How often each order appears in one bootstrap sample of m = 1,000 orders. (b) The probability that an order is never drawn tends to e^{-1} as m grows.

A tree treats a repeated order as several identical orders, so it pulls harder on the tree's decisions:

- **Choosing a split.** A refused order drawn three times counts as three refused orders when CART measures impurity, so a split that separates it looks better.
- **Voting in a leaf.** If a leaf holds two accepted orders and one refused order drawn three times, the leaf predicts refused, three against two.

So bootstrapping gives every order a **random weight** of 0, 1, 2, 3, … for each tree. Two bagged trees differ both in which orders they saw and in which orders they treated as important; two pasted trees differ only in which orders they saw. Bagged trees are therefore **less correlated** (smaller ρ in (29.6)), which lowers the floor. The price is that each bagged tree is slightly worse (higher bias), because it overreacts to some orders and ignores others.

500 trees on 1,000 orders	Sampling	Accuracy on new orders
Single tree	–	71.8%
Pasting (63% of orders)	without replacement	79.2%
Bagging	with replacement	**79.6%**

Two points are worth stressing to students. First, duplicates do not help a **single** tree; they help the **team**. Second, pasting must use a sample smaller than the training set: with `bootstrap=False` and `max_samples=1.0`, every tree sees every order once, all trees are identical, and $\rho = 1$. Bagging usually gives better models and is preferred, but with spare time and CPU it is worth cross-validating both.

29.14 Bagging and pasting in scikit-learn

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    max_samples=100, bootstrap=True, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)           # accuracy 0.799
```

- `n_estimators=500`: train 500 trees.

- `max_samples=100`: each tree sees 100 orders. An integer is a count; a float such as 0.5 is a fraction of the training set.
- `bootstrap=True` gives bagging; `bootstrap=False` gives pasting.
- `n_jobs=-1`: use all CPU cores.

If the base model has `predict_proba()`, as decision trees do, `BaggingClassifier` automatically uses soft voting. The trees do not depend on each other, so they can be trained **in parallel** on different CPU cores or even different servers, and predictions can be made in parallel too. This is why bagging scales well.

29.15 Out-of-bag evaluation

How many orders are left out?

The training set has m orders, and a bootstrap sample makes m independent draws. One draw misses a given order with probability $1 - 1/m$, so all m draws miss it with probability $(1 - 1/m)^m$. As m grows,

$$\lim_{m \to \infty} (1 - 1/m)^m = e^{-1} \approx 0.3679 \quad (29.7)$$

So each tree sees about **63.2%** of the distinct orders. The other **36.8%** are its **out-of-bag (OOB)** orders, and they are a different 37% for each tree. For $m = 10$ the probability is already 0.349, and for $m = 100$ it is 0.366 (Figure 29.9(b)).

THINK FIRST

Every tree in a bagging ensemble never sees about 37% of the training orders. Is this wasted data, or can we use it?

Answer. We can use it. A tree has never seen its OOB orders, so they are fresh test data **for that tree**. For each training order, let only the trees that did **not** train on it vote, and compare the vote with the true label. Repeating this for every order gives an honest estimate of accuracy on new orders without setting aside a validation set.

A worked example

Six trees were trained on bootstrap samples of the ten orders A–J. Order G (truly refused) was out of bag for three trees, T2, T3 and T4. T2 and T3 predict refused and T4 predicts accepted, so the OOB prediction for G is **refused**, which is correct. The other three trees trained on G and are not allowed to vote on it. Doing this for all ten orders gives 7 correct out of 10 (one order, J, ended in a tie). With ten orders and six trees the estimate is rough; with 1,000 orders and 500 trees, each order has about 184 OOB trees and the estimate is reliable.

OOB evaluation in scikit-learn

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    oob_score=True, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)

bag_clf.oob_score_                 # 0.787  (estimate)
bag_clf.score(X_test, y_test)      # 0.796  (5,000 new orders)

bag_clf.oob_decision_function_[:2] # P(accepted), P(refused) for the first
                                   # two training orders, from OOB trees only
```

The OOB estimate (78.7%) is within one percentage point of the test accuracy (79.6%), and it cost no extra data. Figure 29.10 shows that the estimate tracks the test accuracy as trees are added. With fewer than about ten trees, some orders have no OOB tree at all, so the estimate starts at ten trees.

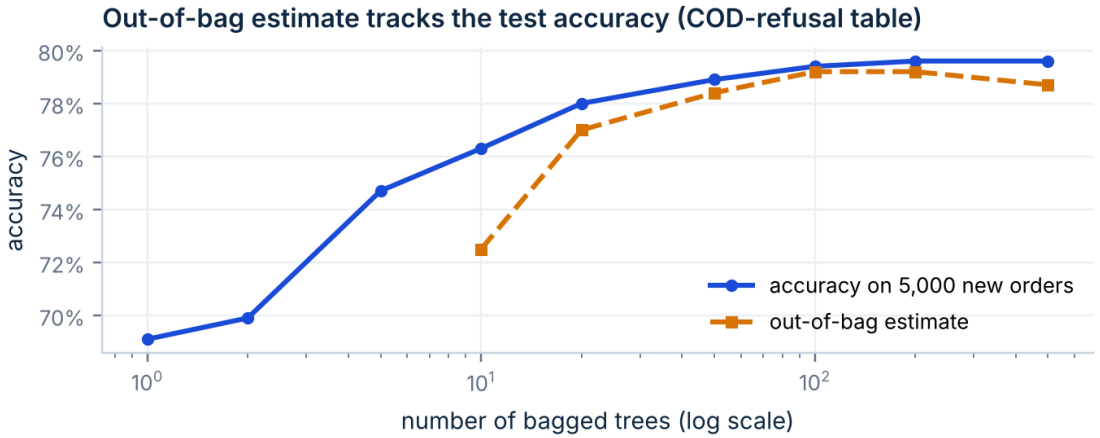


Figure 29.10. Out-of-bag estimate and test accuracy against the number of bagged trees on the COD-refusal table.

29.16 Random patches and random subspaces

`BaggingClassifier` can also sample the **columns**. The hyperparameters `max_features` and `bootstrap_features` work like `max_samples` and `bootstrap`, but for features, so each tree sees a random subset of the nine columns.

Method	Rows (orders)	Columns (features)	Settings	Accuracy
Bagging	sampled	all	`bootstrap=True`	79.6%
Random Subspaces	all	sampled	`bootstrap=False, max_samples=1.0, max_features=0.5`	77.4%
Random Patches	sampled	sampled	`max_samples=0.7, max_features=0.5`	79.0%

Sampling features trades a bit more bias for lower variance, and it is most useful with **many** features, such as images. With only nine columns, two of them strong, hiding half the columns often hides the best ones, and here it did not help. As always, measure.

Lecture 29 · Check yourself

THINK FIRST

Four models vote. Their probabilities of "refused" are 0.9, 0.4, 0.45 and 0.45. What do hard and soft voting predict?

Answer. **Hard:** one vote for refused and three for accepted, so accepted. **Soft:** the average is $2.2/4 = 0.55$, so refused.

THINK FIRST

Why does adding more bagged trees eventually stop helping?

Answer. The variance of the average falls towards the floor $\rho\sigma^2$ of (29.6). More trees cannot reduce the correlation between trees.

THINK FIRST

Why can OOB evaluation replace a validation set for bagging?

Answer. Each order is judged only by trees that **never trained on it**, so the estimate behaves like accuracy on unseen data.

Lecture 30 · Forests and Focus: Random Forests and AdaBoost

Thursday 29th October 2026. We add a second source of randomness to bagged trees and obtain the Random Forest and Extra-Trees. We then ask which features matter, and finally meet a different idea, boosting, through AdaBoost.

Recap of Lecture 29

THINK FIRST
What is the difference between bagging and pasting?

Answer. Bagging samples orders **with** replacement (bootstrap); pasting samples **without** replacement.

THINK FIRST
About what fraction of the training orders is out of bag for one tree?

Answer. About $e^{-1} \approx 0.37$, that is 37%. Each tree sees about 63% of the distinct orders.

THINK FIRST
In $\mathrm{Var}(\bar{h}) = \rho\sigma^2 + (1-\rho)\sigma^2/N$, which term do more trees remove?

Answer. The second. The floor $\rho\sigma^2$ stays unless the trees become **less correlated**, which is the first idea of today.

30.1 The Random Forest

A **Random Forest** is an ensemble of decision trees, generally trained by bagging (sometimes pasting), with `max_samples` set to the size of the training set. Scikit-Learn provides `RandomForestClassifier` (and `RandomForestRegressor`), which is more convenient and optimised for trees than wrapping a tree in `BaggingClassifier`.

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
                                 n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
y_pred_rf = rnd_clf.predict(X_test)       # accuracy 0.798
```

The hyperparameters come in two groups. From `DecisionTreeClassifier` come those that control how each tree grows (`max_depth`, `min_samples_leaf`, `max_leaf_nodes`, …). From `BaggingClassifier` come those that control the ensemble (`n_estimators`, `bootstrap`, `max_samples`, `oob_score`, `n_jobs`). With 500 full-depth trees the forest scores **80.3%** on new orders, with an OOB estimate of 80.1%. A forest, however, is more than bagged trees: it adds one more source of randomness.

30.2 Why do bagged trees still look alike?

THINK FIRST

We bagged 500 trees on the 1,000 orders. Each saw a different sample, yet the trees are still quite alike (correlation 0.36). Look at the first question each tree asks. What do you expect to find?

Answer. **491 of the 500** bagged trees ask the same first question, "delivery days ≤ about 3?". Delivery days is the strongest column, so every bootstrap sample prefers it at the root, and the top of every tree looks the same. One dominant feature makes the trees correlated, and correlation sets the floor $\rho\sigma^2$.

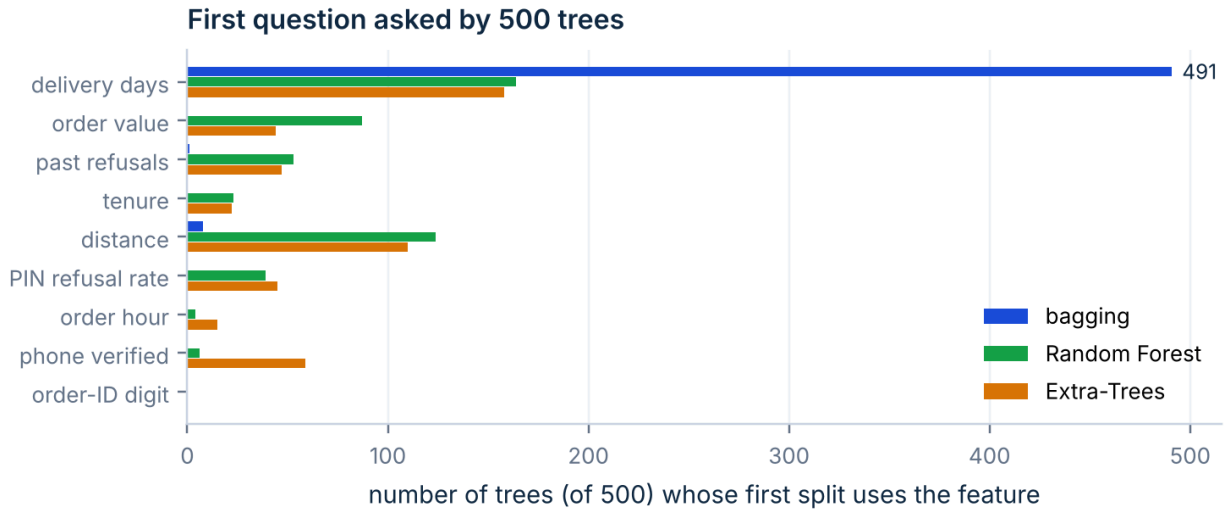


Figure 30.1. Feature used by the first split of each of 500 trees on the COD-refusal table. Bagged trees almost always split first on delivery days; forests and Extra-Trees spread the first question over many features.

A feature lottery at every node

When a Random Forest grows a tree, it does not search for the best feature among **all** features at a node. It draws a **random subset** of features and searches for the best split among those only. By default it draws $\sqrt{n}$ features for classification, where n is the number of features; for our nine columns, $\sqrt{9} = 3$. A fresh subset is drawn at **every node**.

At the root, delivery days gives the largest decrease in Gini impurity (0.0615), followed by distance (0.0513) and order value (0.0260). Delivery days wins the root only when it is among the three drawn features. The probability of that is

$$1 - C(8, 3) \,/\, C(9, 3) = 1 - 56/84 = 1/3$$

so about two-thirds of the trees must start with a different question. In our forest, 164 trees start with delivery days, 124 with distance, 87 with order value, and so on.

Less alike, slightly weaker, better together

500 trees	Accuracy of one tree	Correlation ρ between trees	Accuracy of the ensemble
Bagging	70.1%	0.355	79.6%
Random Forest	68.9%	0.307	**80.3%**

500 trees	Accuracy of one tree	Correlation ρ between trees	Accuracy of the ensemble
Extra-Trees	68.3%	0.288	80.0%

The random feature subsets make each tree a little **worse** (higher bias) but **less alike** (lower ρ). The floor $\rho\sigma^2$ drops, and the forest beats plain bagging. This is the same trade we met with bootstrapping: a little more bias for a lower variance.

30.3 A forest is almost a bag of random trees

The following `BaggingClassifier` is roughly equivalent to the Random Forest of Section 30.1:

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500, max_samples=1.0, bootstrap=True,
    n_jobs=-1, random_state=42)
# accuracy 0.794   (RandomForestClassifier: 0.798)
```

The key argument is `max_features="sqrt"`: a fresh random subset of features at every node.

> **A correction to last year's slides**
> Last year's slide used `splitter="random"`. That option does something different: it draws **random thresholds** and still considers **all** features at every node. It is closer to Extra-Trees (Section 30.5). On our orders it scores 78.2%. The dedicated `RandomForestClassifier` is also faster and supports `oob_score`, `class_weight` and `feature_importances_` directly.

30.4 How many trees?

Adding trees lowers the variance of a forest and never makes it overfit; the accuracy simply levels off (Figure 30.2). Beyond that point, more trees only cost time.

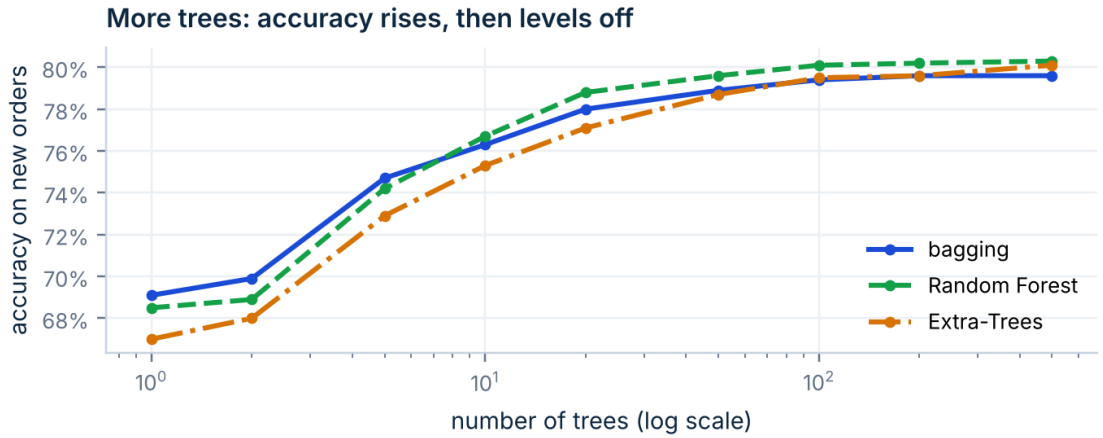


Figure 30.2. Accuracy on 5,000 new orders against the number of trees for bagging, Random Forest and Extra-Trees.

Hyperparameter	Effect
`n_estimators`	More trees: lower variance, slower. Accuracy levels off; it does not overfit.
`max_features`	Smaller: trees less alike but individually weaker. The main tuning knob.

Hyperparameter	Effect
`min_samples_leaf`, `max_depth`	Regularise each tree.
`oob_score`	A free validation estimate.
`class_weight`	Rebalances rare classes.

30.5 Extremely Randomised Trees (Extra-Trees)

A normal tree, and a Random Forest, tries **every** threshold of each candidate feature and keeps the one with the lowest weighted impurity J (Lecture 27). **Extra-Trees** (extremely randomised trees) draws **one random threshold** for each candidate feature and keeps the best of these.

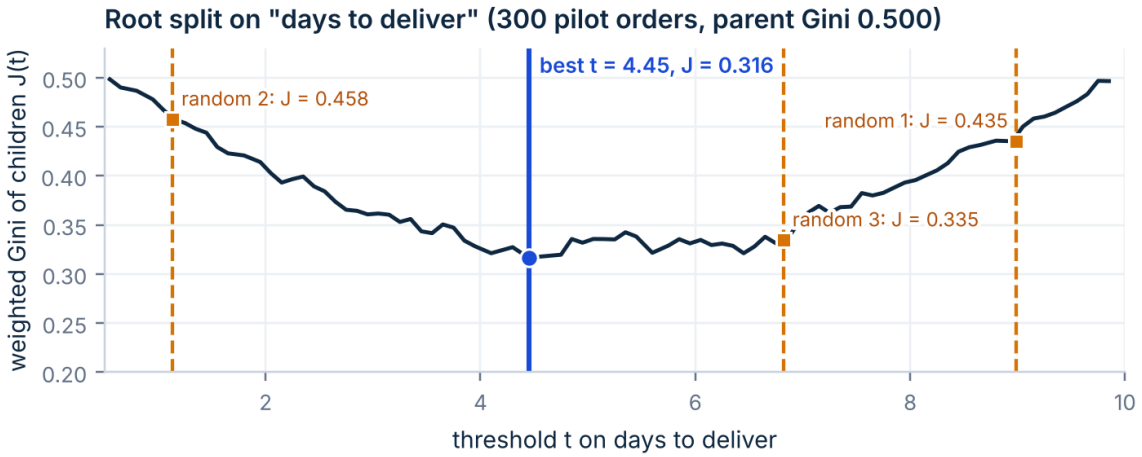


Figure 30.3. Weighted Gini impurity J(t) of the children for every threshold on "days to deliver" in the pilot orders. CART picks the best threshold (blue); Extra-Trees draws random thresholds (amber), which are usually worse.

- Even more diversity: a little more bias, lower variance.
- Much **faster** to train, because searching for the best threshold is the slowest part of growing a tree.
- By default, Extra-Trees does not bootstrap: each tree sees all the orders. Its diversity comes from the random features and thresholds.

```
from sklearn.ensemble import ExtraTreesClassifier
ext_clf = ExtraTreesClassifier(n_estimators=500, n_jobs=-1, random_state=42)
ext_clf.fit(X_train, y_train)            # accuracy 0.800
```

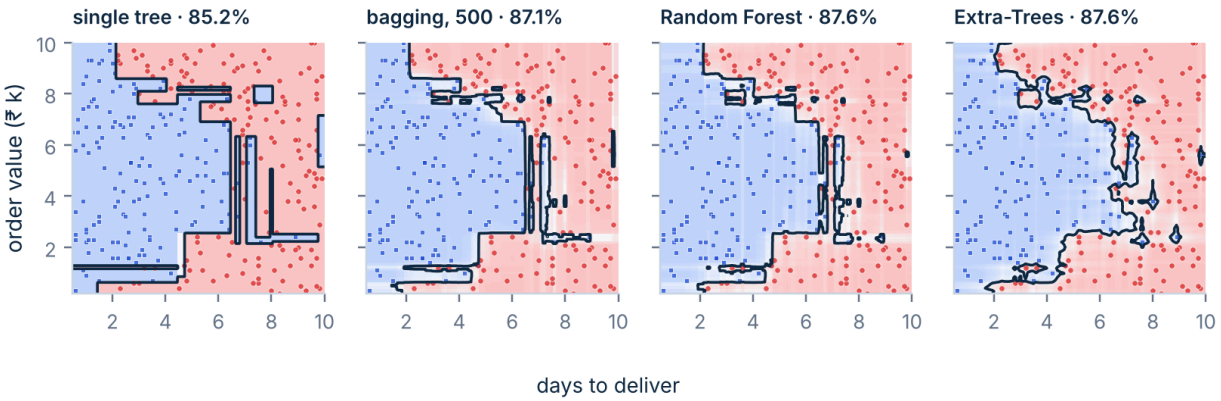


Figure 30.4. Four models on the 300 pilot orders with their accuracy on 2,000 new pilot orders.

1,000 orders, 9 columns	Single tree	Bagging	Random Forest	Extra-Trees
Accuracy on new orders	71.8%	79.6%	**80.3%**	80.0%
Fit time, 500 trees (relative)	–	2.7×	1.3×	1.0×

THINK FIRST

Extra-Trees is faster and more random; a Random Forest searches harder. Which one should ShopKart use?

Answer. It is hard to tell in advance. Compare them with **cross-validation**, tuning the hyperparameters with a grid search. On our orders the tuned forest wins narrowly; on another dataset Extra-Trees may win.

```
from sklearn.model_selection import GridSearchCV
grid = {"max_features": [2, 3, 5, 9], "min_samples_leaf": [1, 3, 5, 10]}
for model in (RandomForestClassifier(300), ExtraTreesClassifier(300)):
    search = GridSearchCV(model, grid, cv=5)
    search.fit(X_train, y_train)
```

Model	Best settings	CV accuracy	Test accuracy
Random Forest	`max_features=3, min_samples_leaf=3`	79.8%	**80.4%**
Extra-Trees	`max_features=5, min_samples_leaf=1`	79.1%	79.5%

30.6 Feature importance

Mean decrease in impurity (MDI)

Scikit-Learn measures a feature's importance by how much the nodes that use it **reduce impurity**, averaged over all trees. Each node is weighted by its number of training samples. For feature k,

$$\text{imp}(k) \propto \sum_{\text{nodes that split on } k} \left(n \cdot G - n_L \cdot G_L - n_R \cdot G_R \right) \quad (30.1)$$

where n and G are the number of samples and the Gini impurity at the node, and n_L, G_L, n_R, G_R are the same for its left and right children. The sum is averaged over trees and then scaled so that all importances sum to 1. The scores are in the attribute `feature_importances_`.

Check with Lecture 28's payment tree

The depth-2 payment-method tree of Lecture 28 has two splits:

- root, on tenure: $150 \times 0.667 - 50 \times 0 - 100 \times 0.5 = 50.0$;
- node 2, on order value: $100 \times 0.5 - 54 \times 0.226 - 46 \times 0.122 = 32.2$.

So tenure scores $50.0 / 82.2 = 0.608$ and order value $32.2 / 82.2 = 0.392$, exactly the importances printed in Lecture 28.

Which columns predict a refusal?

```
rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
rnd_clf.fit(X_train, y_train)
for name, score in zip(feature_names, rnd_clf.feature_importances_):
    print(name, round(score, 3))
```

THINK FIRST

The last digit of the order ID is pure noise, yet its MDI is 0.053, higher than "phone verified" (0.032). Should we keep it?

Answer. No. MDI is computed on the **training** data, and it favours features with **many distinct values**: a deep tree can always find some threshold on a noisy column that tidies a few training orders. "Phone verified" has only two values, so it gets few chances. MDI also splits the credit between **correlated** columns, such as delivery days and distance. We need a better test.

Permutation importance

Idea: on **new** orders, shuffle one column at random, which breaks its link with the outcome, and measure how much the accuracy drops. A useful column leaves a large drop; a useless one changes nothing.

```
from sklearn.inspection import permutation_importance
r = permutation_importance(rnd_clf, X_test, y_test, n_repeats=10, random_state=42)
r.importances_mean                          # drop in accuracy for each column
```

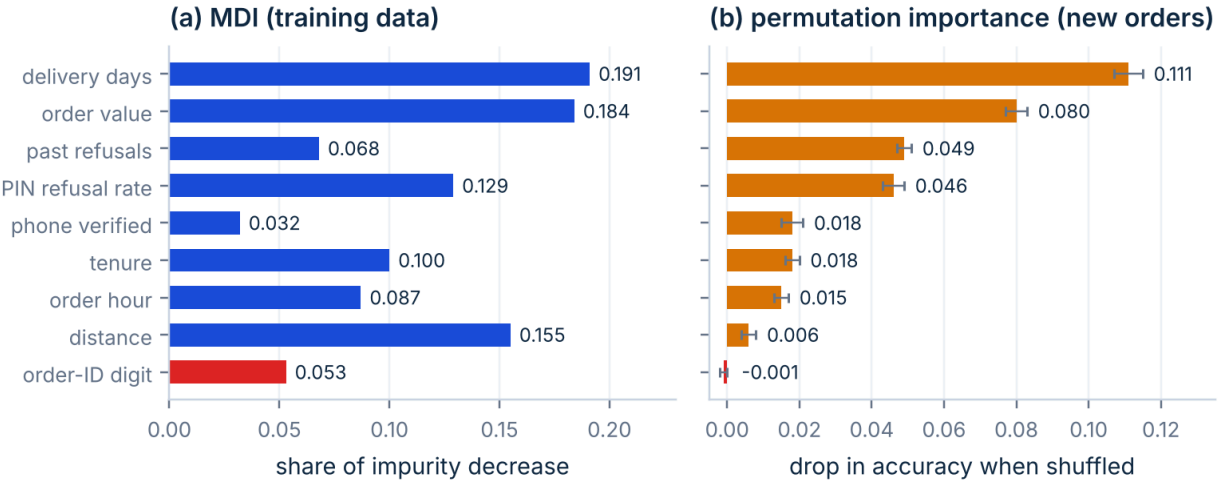


Figure 30.5. MDI computed on the training orders (a) and permutation importance on 5,000 new orders (b). The order-ID digit (red) looks useful to MDI but changes nothing when shuffled.

Permutation importance gives the order-ID digit a score of −0.001: it is useless. Distance also drops sharply, because delivery days already carries most of its information. Dropping the order-ID column changes the forest's test accuracy from 80.3% to **80.7%**: the column was only adding noise.

Which pixels read a PIN code?

ShopKart reads handwritten PIN codes on return labels. A Random Forest trained on the 1,797 scanned digits of 8 × 8 pixels from Lecture 20–21 reaches 94.1% five-fold cross-validated accuracy. Plotting each pixel's importance (Figure 30.6) shows that border pixels are almost never used, because they are blank in every digit; the bright pixels are where digits **differ** from each other. Random Forests give a quick view of which inputs matter, which helps with **feature selection**.

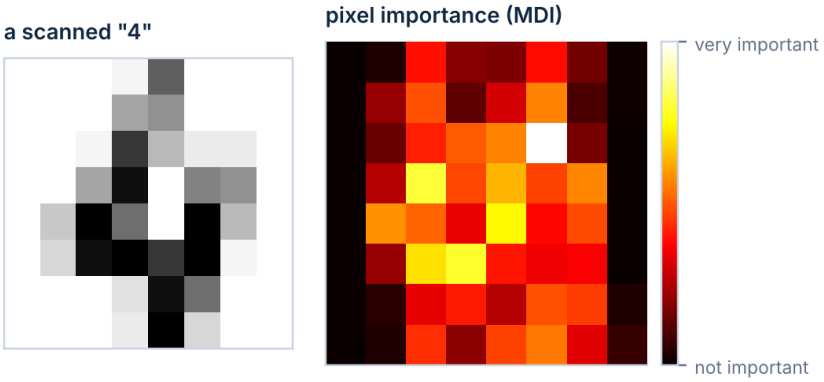


Figure 30.6. A scanned digit and the MDI importance of each of its 64 pixels for a Random Forest of 500 trees.

30.7 Boosting

THINK FIRST

You took a mock test and got 12 of 50 questions wrong. You have time for one more round of revision. How should you spend it?

Answer. Spend most of it on the **questions you got wrong**, while keeping a light eye on the rest. After the next mock test, focus on what is **still** wrong. This is boosting.

Boosting (originally *hypothesis boosting*) is any ensemble method that combines several weak learners into a strong learner. Most boosting methods train predictors **sequentially**, each trying to correct its predecessor. The two most popular are **AdaBoost** (Adaptive Boosting), which re-weights the training orders, and **Gradient Boosting** (Lecture 31), which fits the remaining errors.

30.8 AdaBoost

AdaBoost pays more attention to the training orders that its predecessors got wrong:

1. Train a first base classifier, for example a **decision stump** (a tree of depth 1), with equal weights on all orders.
2. Predict the training orders.
3. **Increase the weight** of the misclassified orders.

4. Train the next classifier on the **re-weighted** orders.
5. Repeat. Finally, all classifiers vote, the better ones with a bigger say.

Each round needs the previous round's result, so boosting cannot be parallelised (or only partly). It does not scale as well as bagging.

The equations

Each order starts with weight $w^{(i)} = 1/m$. The **weighted error rate** of the j-th predictor is the share of weight on the orders it gets wrong:

$$r_j = \left(\sum_{i \,:\, \hat{y}_j^{(i)} \neq y^{(i)}} w^{(i)} \right) / \left(\sum_{i=1}^{m} w^{(i)} \right) \tag{30.2}$$

Its **predictor weight**, its say in the final vote, is

$$\alpha_j = \eta \cdot \log\left((1 - r_j) / r_j \right) \tag{30.3}$$

where η is the learning rate (default 1). A good predictor (small r_j) gets a large α_j; a predictor no better than a coin ($r_j = 0.5$) gets $\alpha_j = 0$; a predictor worse than a coin gets a negative weight, so its vote is reversed (Figure 30.7).

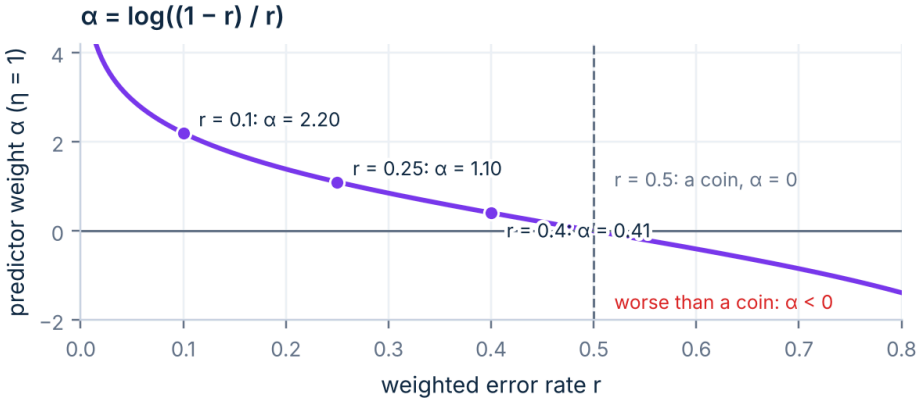


Figure 30.7. Predictor weight α against weighted error rate r, with η = 1.

After predictor j, the misclassified orders are **boosted**:

$$w^{(i)} \leftarrow w^{(i)} \cdot \exp(\alpha_j) \quad \text{if } \hat{y}_j^{(i)} \neq y^{(i)}; \quad \text{otherwise } w^{(i)} \text{ is unchanged} \tag{30.4}$$

Then all weights are **normalised** (divided by $\sum_i w^{(i)}$), a new predictor is trained on the new weights, and the process repeats. It stops when the desired number of predictors is reached, or when a perfect predictor is found. To predict, every predictor votes for a class with weight α_j, and the class with the largest total wins:

$$\hat{y}(x) = \arg\max_k \sum_{j \,:\, \hat{y}_j(x) = k} \alpha_j \tag{30.5}$$

30.9 AdaBoost by hand

We use the ten orders A–J again: A, B, C, F and I were accepted; D, E, G, H and J were refused. All weights start at 0.1.

Round 1

The best stump is "days ≤ 3.75 → accepted, otherwise refused". It misclassifies F and I, two accepted orders that took longer than 3.75 days.

Order	Label	w	Stump says	w × e^α if wrong	Normalised
A	accepted	0.100	accepted	0.100	0.0625
B	accepted	0.100	accepted	0.100	0.0625
C	accepted	0.100	accepted	0.100	0.0625
D	refused	0.100	refused	0.100	0.0625
E	refused	0.100	refused	0.100	0.0625
F	accepted	0.100	**refused ✗**	**0.400**	**0.2500**
G	refused	0.100	refused	0.100	0.0625
H	refused	0.100	refused	0.100	0.0625
I	accepted	0.100	**refused ✗**	**0.400**	**0.2500**
J	refused	0.100	refused	0.100	0.0625

- $r_1 = 0.1 + 0.1 = 0.2$
- $\alpha_1 = \log(0.8 / 0.2) = \log 4 = 1.386$
- Boost the wrong orders: $0.1 \times e^{1.386} = 0.1 \times 4 = 0.4$
- Normalise: divide by $8 \times 0.1 + 2 \times 0.4 = 1.6$

F and I now carry half of all the weight (0.25 each), so the next stump must take them seriously.

Rounds 2 and 3, then the vote

Round	Stump	Wrong	r	α	Normalising sum
1	days ≤ 3.75 → accepted	F, I	0.200	1.386	1.600
2	value ≤ 2.1 → refused	E, H, J	0.188	1.466	1.625
3	value ≤ 5.05 → accepted	C, D, G	0.115	2.037	1.769

In round 2, F and I are heavy, so the stump keeps them right and sacrifices three light orders: $r_2 = 3 \times 0.0625 = 0.1875$ and $\alpha_2 = \log(0.8125 / 0.1875) = 1.466$. In round 3 the heaviest orders are E, H and J (0.167 each) and F and I (0.154 each); the stump gets all of them right and misses C, D and G, which weigh only 0.0385 each, so $r_3 = 0.115$ and $\alpha_3 = 2.037$.

For the new order N (6 days, ₹5,000): stump 1 says refused ($\alpha = 1.39$), stump 2 says accepted ($\alpha = 1.47$) and stump 3 says accepted ($\alpha = 2.04$). The totals are 3.50 for accepted and 1.39 for refused, so AdaBoost predicts **accepted**. The three stumps together classify all ten training orders correctly, although each one alone gets two or three wrong.

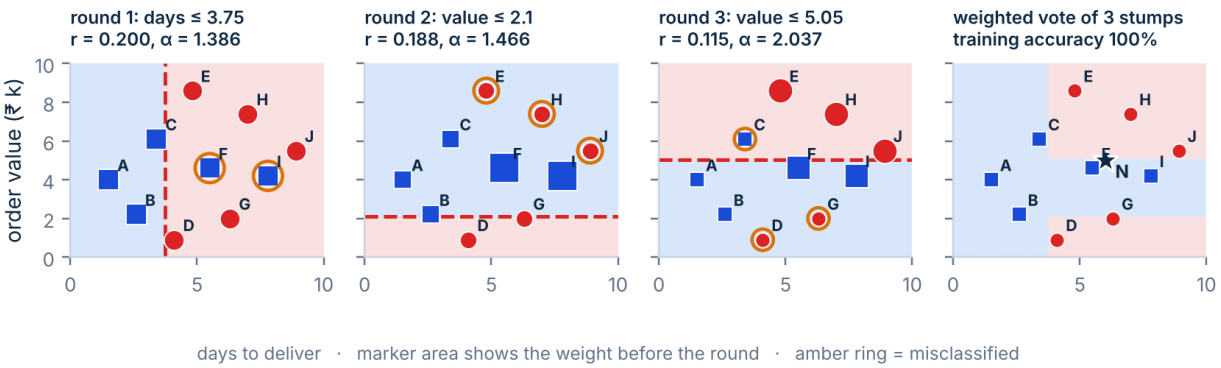


Figure 30.8. Three rounds of AdaBoost on the ten orders. Marker area shows each order's weight before the round; amber rings mark the stump's mistakes. Right: the weighted vote of the three stumps.

AdaBoost on the pilot orders

Figure 30.9 shows AdaBoost with stumps on the 300 pilot orders. Each stump draws one straight line and is weak on its own (78.3% on new orders). As rounds are added, the weights collect on the hard orders near the boundary, and the weighted vote builds a curved, staircase-like boundary. After 150 stumps the ensemble scores 88.4% on new orders.

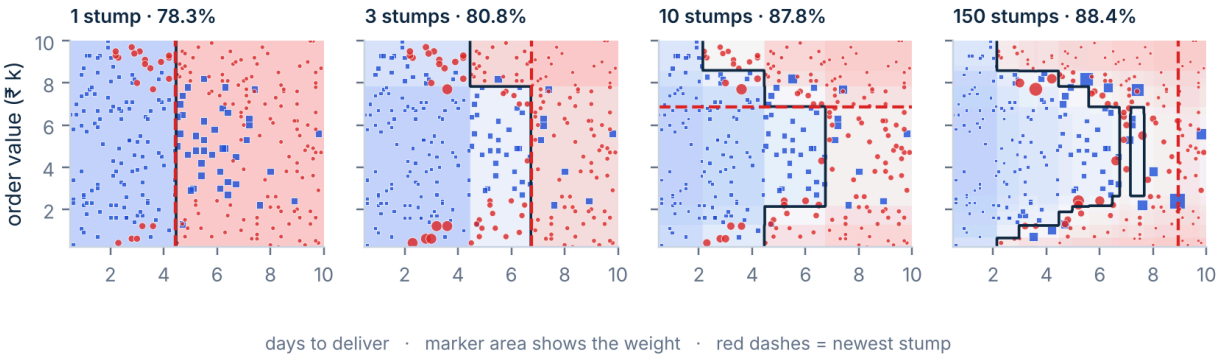


Figure 30.9. AdaBoost with decision stumps on the pilot orders after 1, 3, 10 and 150 rounds, with accuracy on 2,000 new orders. Marker area shows the current weight of each order.

30.10 AdaBoost in scikit-learn

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=200,
    learning_rate=0.5, random_state=42)
ada_clf.fit(X_train, y_train)
ada_clf.score(X_test, y_test)              # 0.826
```

On the COD-refusal table, one stump ("days ≤ 3.25") scores 61.6%, while 200 boosted stumps score **82.6%**: weak learners have become a strong learner.

Scikit-Learn uses a multiclass version of AdaBoost called **SAMME** (Stagewise Additive Modelling using a Multiclass Exponential loss function). With K classes, the predictor weight becomes

$$\alpha_j = \eta \cdot \left(\log\left((1 - r_j) / r_j \right) + \log(K - 1) \right) \quad (30.6)$$

For two classes, $\log(K-1) = \log 1 = 0$, so SAMME is exactly AdaBoost.

An update to last year's slides

`algorithm="SAMME.R"` was deprecated in Scikit-Learn 1.6 and has since been removed. In version 1.9, passing `algorithm` raises a `TypeError`. Simply leave it out.

THINK FIRST

A few orders in the training data are mislabelled: the customer accepted the parcel, but the system recorded a refusal. What will AdaBoost do with them, round after round?

Answer. No honest stump can classify a mislabelled order correctly, so it is misclassified again and again, and its weight grows **every round**. Later stumps bend to fit these few wrong labels: AdaBoost **overfits noisy labels**. Remedies: fewer estimators, a smaller `learning_rate`, simpler base learners, and cleaning the labels.

Bagging versus boosting

Bagging trains its members in **parallel** and mainly reduces **variance**. Boosting trains them in **sequence** and mainly reduces **bias**: weak stumps become a strong model.

Lecture 30 · Check yourself

THINK FIRST

What does a Random Forest add to bagged trees, and why does it help?

Answer. A random subset of features at **each node**. The trees become less correlated, which lowers the variance floor $\rho\sigma^2$.

THINK FIRST

A stump has weighted error 0.25. What is its α (with η = 1)? By what factor do the weights of its mistakes grow?

Answer. $\alpha = \log 3 \approx 1.099$. Its mistakes are multiplied by $e^{\alpha} = 3$ before normalising.

THINK FIRST

Why should we prefer permutation importance over MDI for the order-ID column?

Answer. MDI is measured on training data and favours many-valued features. Permutation importance on new orders shows that the column adds nothing.

Lecture 31 · Fixing the Errors: Gradient Boosting and Stacking

Friday 30th October 2026. Gradient Boosting adds trees that predict what the ensemble still gets wrong. Stacking trains a model to combine other models. We close with a comparison of every method on the same ShopKart problem.

Recap of Lecture 30

THINK FIRST
In AdaBoost, what changes from one round to the next?

Answer. The **weights of the training orders**. Misclassified orders become heavier, so the next stump focuses on them.

THINK FIRST
A stump with weighted error 0.5 gets what α?

Answer. $\alpha = \log(0.5 / 0.5) = 0$. It is no better than a coin, so it has no say.

THINK FIRST
Bagging mainly reduces variance. What does boosting mainly reduce?

Answer. **Bias.** Weak, simple learners are added one by one until the ensemble fits the pattern.

31.1 The idea: predict the remaining error

We return to the delivery-time problem of Lecture 28: 200 past orders, with distance (km) as the input and delivery time (hours) as the target. Lecture 28's depth-2 regression tree predicts 13.48 hours for every order up to 169.5 km.

THINK FIRST
For orders between 70 and 170 km, the depth-2 tree is about five hours too low. We may not change this tree. What could we add to it?

Answer. Train a **second** model whose job is to predict the **error** the first tree makes, the **residual** $y - h_1(x)$, and **add** the two predictions. Then a third model predicts what is **still** wrong, and so on. This is **Gradient Boosting**.

Like AdaBoost, Gradient Boosting adds predictors one after another, each correcting its predecessor. Instead of changing order weights, it fits each new predictor to the **residual errors** of the ensemble so far. With decision trees as base predictors it is called **Gradient Tree Boosting** or **Gradient Boosted Regression Trees (GBRT)**.

31.2 Three trees by hand

```
from sklearn.tree import DecisionTreeRegressor

tree_reg1 = DecisionTreeRegressor(max_depth=2)
tree_reg1.fit(X, y)

y2 = y - tree_reg1.predict(X)             # residuals of tree 1
tree_reg2 = DecisionTreeRegressor(max_depth=2)
tree_reg2.fit(X, y2)

y3 = y2 - tree_reg2.predict(X)            # what is still wrong
tree_reg3 = DecisionTreeRegressor(max_depth=2)
tree_reg3.fit(X, y3)

y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```

Figure 31.1 shows the process. In the left column, each tree is trained on the residuals of the trees before it. In the right column, the ensemble's prediction is the sum of the trees so far. In the first row the ensemble has one tree, so its predictions equal tree 1's. In each later row a new tree is trained on the remaining residuals, and the predictions gradually improve.

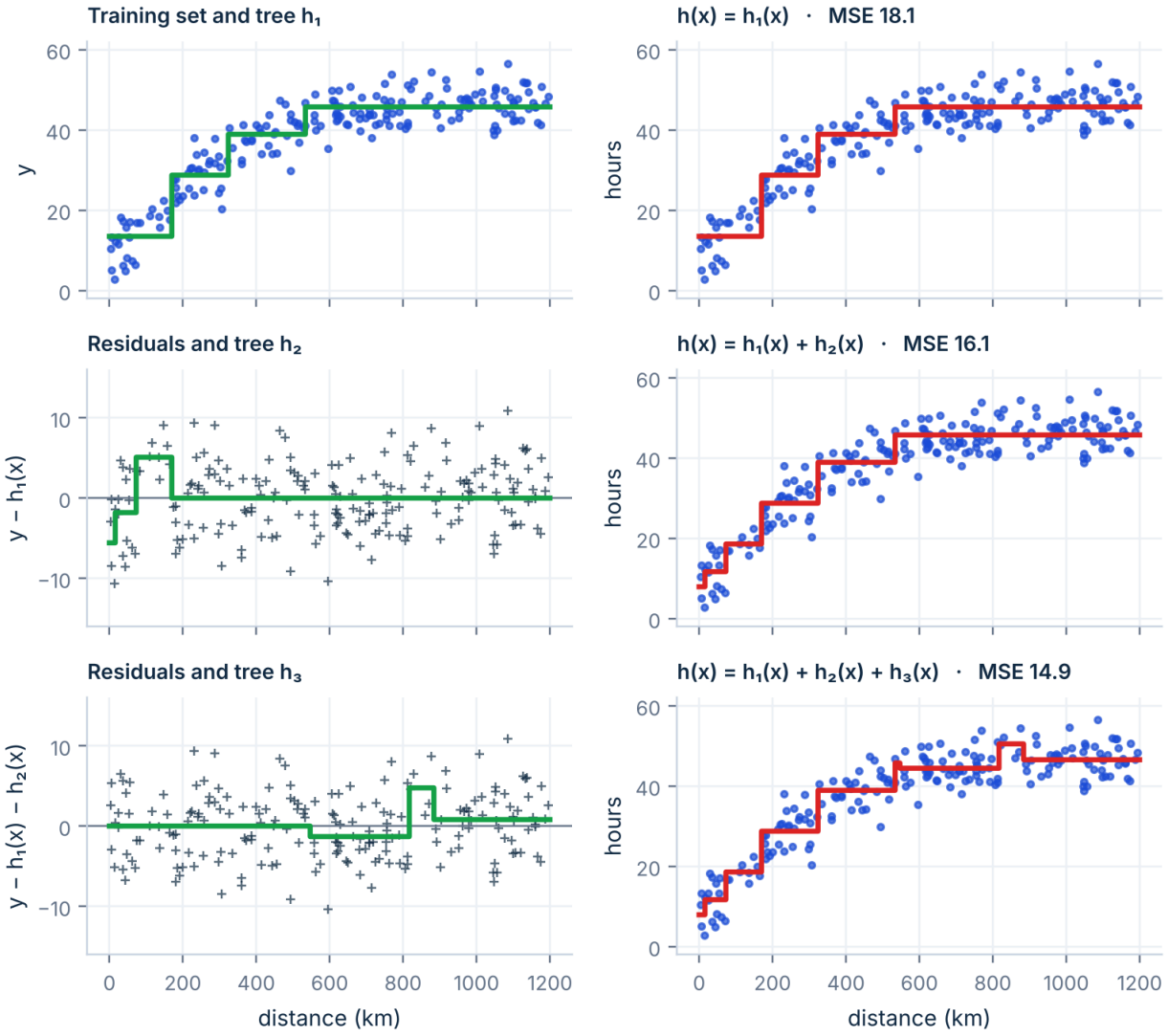


Figure 31.1. Gradient boosting with three depth-2 trees on 200 delivery orders. Left: the target (row 1) or the residuals (rows 2 and 3) with the tree fitted to them (green). Right: the sum of the trees so far (red) and its training MSE.

Trees in the ensemble	Training MSE (hours²)
tree 1	18.1
tree 1 + tree 2	16.1
tree 1 + tree 2 + tree 3	**14.9**

Worked example

For an order 150 km away, tree 1 predicts 13.48 h, tree 2 adds 5.09 h and tree 3 adds 0.01 h:

$$\hat{y} = 13.48 + 5.09 + 0.01 = 18.58 \text{ hours}$$

Tree 2 learnt exactly the correction the signpost question asked for. For an order 850 km away, the trees give $45.73 + 0.00 + 4.77 = 50.50$ hours.

31.3 Why is it called gradient boosting?

Take the squared-error loss for one order, with current ensemble prediction $F(x_i)$:

$$L_i = \tfrac{1}{2} \cdot (\, y_i - F(x_i)\,)^2$$

Its derivative with respect to the prediction is $\partial L_i / \partial F(x_i) = -(\, y_i - F(x_i)\,)$, so

$$-\,\partial L_i / \partial F(x_i) = y_i - F(x_i) \quad (31.1)$$

The **residual is the negative gradient** of the loss with respect to the prediction. Fitting a tree to the residuals and adding it is therefore a **gradient-descent step**, taken not on model weights (as in Lecture 13–14) but on the **predictions themselves**.

The algorithm

1. **Start** with a constant: $F_0(x) = \bar{y}$, the mean delivery time (37.98 hours).
2. **Compute the residuals** $r_i = y_i - F_{m-1}(x_i)$ for every training order.
3. **Fit** a small tree h_m to the pairs (x_i, r_i).
4. **Step**: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, then repeat from step 2.

For other loss functions only step 2 changes: r_i becomes the negative gradient of that loss. For the log loss used in classification it is $y_i - \hat{p}_i$, the true label minus the predicted probability.

31.4 Gradient Boosting in scikit-learn

This creates the **same** ensemble as the three trees by hand:

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3, learning_rate=1.0)
gbrt.fit(X, y)
gbrt.predict([[150]])                    # 18.58 hours, as by hand
```

With `learning_rate=1.0`, scikit-learn starts from the mean and adds the three trees, which gives exactly our sum (the mean is absorbed into the first tree's leaves). The hyperparameters again come in two groups: those that control the growth of each tree (`max_depth`, `min_samples_leaf`, …) and those that control the ensemble (`n_estimators`, `learning_rate`). With `subsample=0.5`, each

tree is trained on a random half of the orders; this is called **Stochastic Gradient Boosting** and trades a little more bias for lower variance and faster training.

`GradientBoostingClassifier` uses the log loss. On the COD-refusal table it scores **81.9%** with its default settings.

31.5 Shrinkage

The `learning_rate` η scales each tree's contribution. With a **low** value such as 0.1 we need **more trees** to fit the training set, but the predictions usually **generalise better**. This regularisation technique is called **shrinkage**.

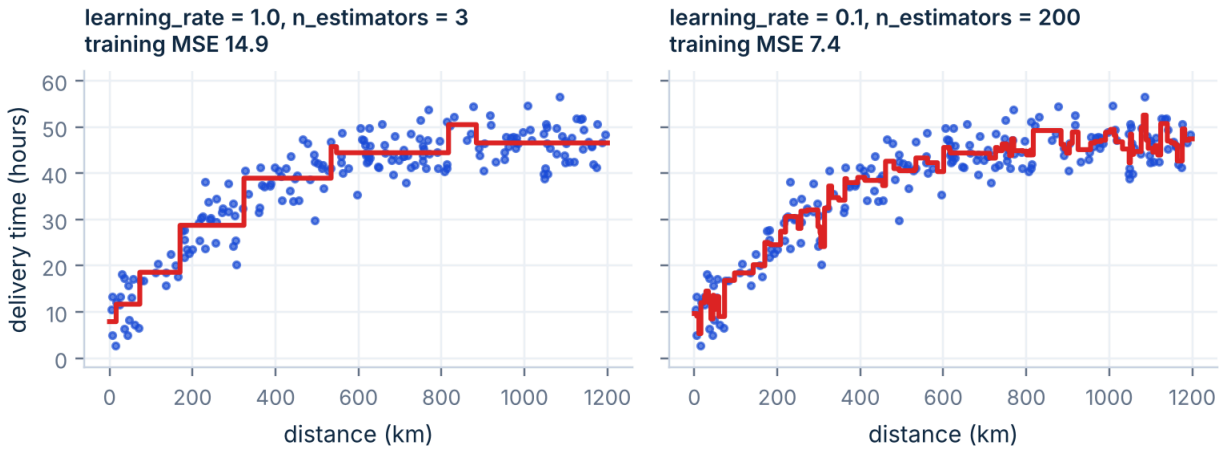


Figure 31.2. Gradient boosting on the delivery orders with a large learning rate and few trees (left) and a small learning rate and many trees (right).

With $\eta = 1.0$ and 3 trees the training MSE is 14.9; with $\eta = 0.1$ and 200 trees it is 7.4. The second model fits the training orders more closely, perhaps too closely: its prediction has small wiggles that follow individual orders. The number of trees must therefore be chosen with care.

THINK FIRST

With η = 0.1, each tree corrects only 10% of the remaining error, so we need many more trees. Why would this give better predictions on new orders than taking full steps?

Answer. A full step trusts each tree completely, including the noise it fitted. A small step **averages** the contributions of many trees, each fitted to slightly different residuals, so no single tree's mistakes dominate. The ensemble approaches the pattern **gradually**, and we can **stop** as soon as it starts to fit noise. It is the same idea as a small learning rate in gradient descent: slower, but steadier.

31.6 Early stopping: how many trees?

Figure 31.3 shows the training and validation MSE after each tree, with $\eta = 0.1$ and depth-2 trees. The validation MSE is the five-fold cross-validated average on the 200 orders. It falls, reaches a **minimum** at **31 trees** (MSE 19.3), and then rises slowly as the trees start fitting noise, while the training MSE keeps falling.

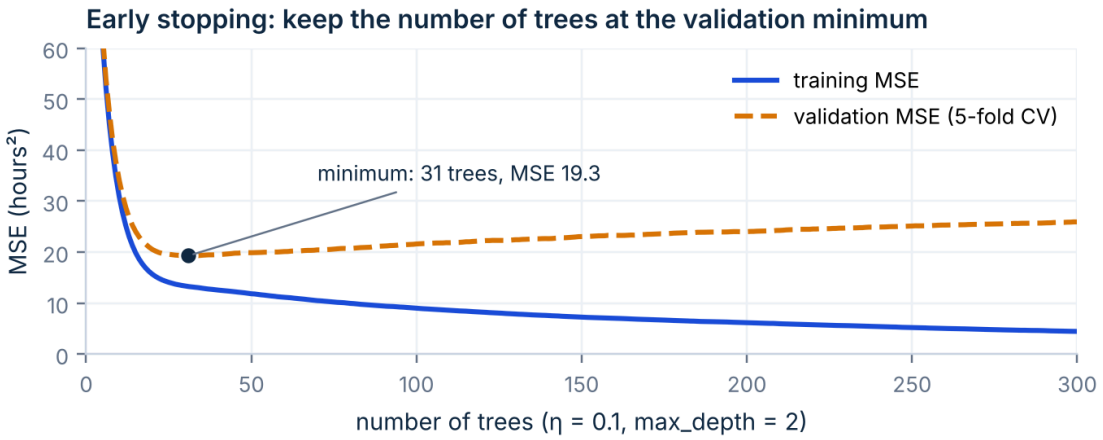


Figure 31.3. Training and cross-validated MSE against the number of trees. The best number of trees is at the validation minimum.

For comparison, a single depth-2 tree has a cross-validated MSE of 21.7 and a depth-3 tree 19.7. Scikit-Learn can stop automatically when the validation score stops improving:

```
gbrt = GradientBoostingRegressor(
    max_depth=2, learning_rate=0.1,
    n_estimators=1000,            # an upper limit
    n_iter_no_change=10,          # stop after 10 trees without improvement
    validation_fraction=0.25, random_state=42)
gbrt.fit(X, y)
gbrt.n_estimators_                # 44 trees were kept
```

Older code achieves the same with `warm_start=True` and a loop that adds trees one at a time, or with `staged_predict()`, which returns the prediction after each tree so that we can scan all sizes. `n_iter_no_change` does this for us.

31.7 Fast gradient boosting libraries

- **HistGradientBoostingClassifier / Regressor** (Scikit-Learn) bins each feature into at most 255 values, so finding splits is very fast on large datasets. It handles missing values and categorical features natively and uses early stopping by default on large data. On the COD table it scores 79.8%.
- **XGBoost**, **LightGBM** and **CatBoost** are separate, highly optimised libraries. XGBoost stands for *Extreme Gradient Boosting*. They win many competitions on tabular data, and their interfaces follow Scikit-Learn's.

```
from sklearn.ensemble import HistGradientBoostingClassifier
hgb = HistGradientBoostingClassifier(random_state=42)
hgb.fit(X_train, y_train)                # 0.798

# pip install xgboost
import xgboost
xgb = xgboost.XGBClassifier(n_estimators=300, learning_rate=0.05)
xgb.fit(X_train, y_train)
```

With only 1,000 orders the classic `GradientBoostingClassifier` does as well as the fast versions; speed matters with millions of orders.

31.8 Bagging versus boosting

	Bagging · Random Forest	Boosting · AdaBoost, Gradient Boosting
Training	parallel; members independent	sequential; each member depends on the previous ones
Base learner	deep trees: low bias, high variance	shallow trees (stumps, depth 2–5): high bias
Mainly reduces	variance	bias (and variance, with shrinkage)
More members	never hurts; accuracy levels off	can overfit; use early stopping
Noisy labels	robust	sensitive, especially AdaBoost
Key hyperparameters	`n_estimators`, `max_features`	`learning_rate`, `n_estimators`, `max_depth`
COD refusals	Random Forest 80.3%	AdaBoost 82.6%; Gradient Boosting 81.9%

Here boosting wins because the refusal pattern is mostly a **sum of simple effects** (slow delivery, past refusals, a risky PIN code, an unverified phone), which small trees added one by one capture well.

31.9 Stacking

THINK FIRST

In a voting classifier, every member counts equally, according to a fixed rule. But some members are more reliable than others, perhaps only in certain situations. Can we do better than a fixed rule?

Answer. Yes: **train a model** to do the combining. Its inputs are the members' predictions, and its target is the true label. It learns whom to trust, and how much. This is **stacking** (short for *stacked generalisation*, Wolpert, 1992). The combining model is called the **blender** or **meta learner**.

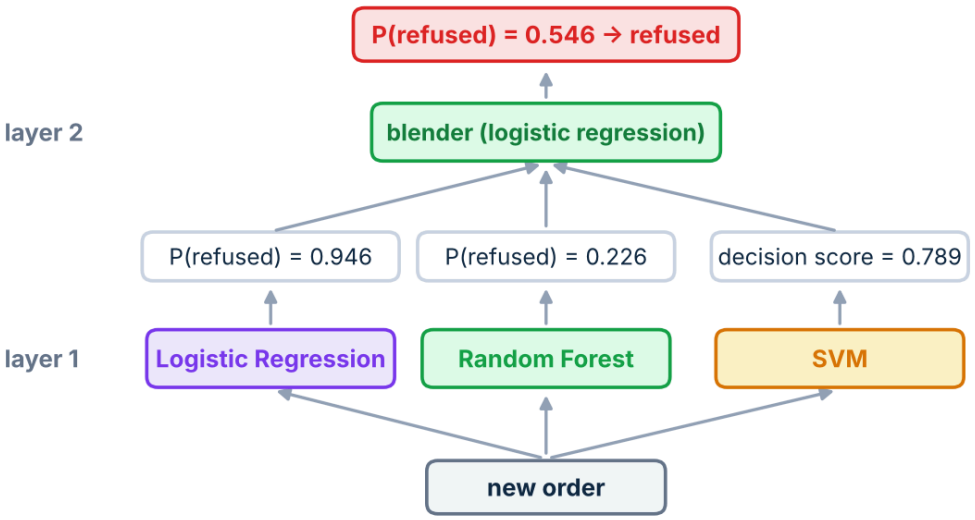


Figure 31.4. Stacking for one new order. Three first-layer models make predictions; the blender combines them into the final probability of refusal.

What the blender learnt

Our blender is a logistic regression. Its inputs are z_1, the probability of refusal from Logistic Regression; z_2, the same from the Random Forest; and z_3, the SVM's decision score. It learnt

$$\hat{p} = \sigma(\, 0.944 \cdot z_1 + 3.073 \cdot z_2 + 0.752 \cdot z_3 - 1.997 \,) \quad (31.2)$$

where $\sigma(t) = 1 / (1 + e^{-t})$ is the sigmoid function of Lecture 11–12. For the order in Figure 31.4:

$$0.944 \times 0.946 + 3.073 \times 0.226 + 0.752 \times 0.789 - 1.997 = 0.183$$

$$\hat{p} = \sigma(0.183) = 0.546 \;\rightarrow\; \text{refused (correct)}$$

The blender trusts the forest most (weight 3.07). Even so, here Logistic Regression's strong opinion (0.946) and the SVM's positive score outweigh the forest's low value. Voting would weight the three members equally; the blender learns **unequal** weights, and a non-linear blender (for example, a Random Forest) could even learn **when** to trust each member.

31.10 Training the blender: the hold-out approach

1. **Split** the 1,000 orders into subset 1 (700 orders) and a hold-out subset 2 (300 orders).
2. **Train** the first-layer models (Logistic Regression, Random Forest, SVM) on subset 1 only.
3. **Predict** subset 2 with them. These predictions are "clean": the models have never seen these orders.
4. Build a new training set: **three columns** of predictions plus the true labels.
5. **Train the blender** on it, so that it learns to predict the label from the first layer's predictions.

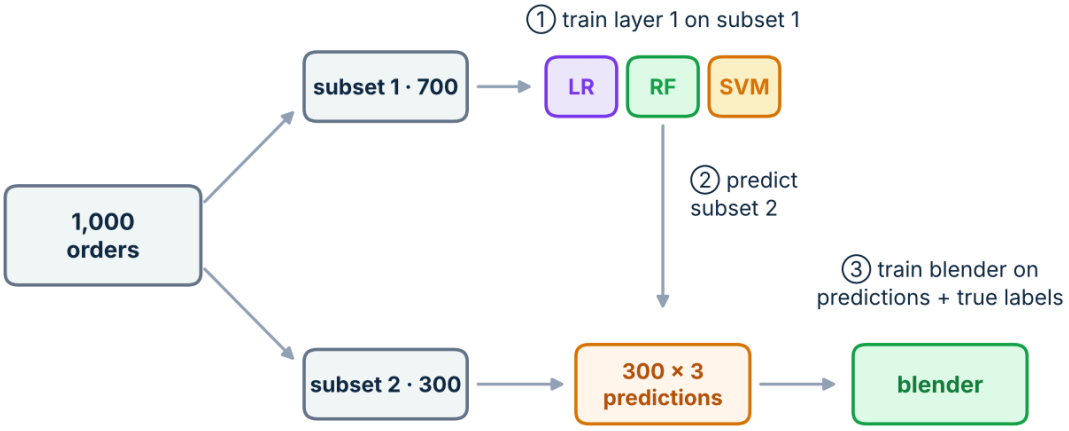


Figure 31.5. Training a blender with a hold-out set.

The blender learnt weights 1.16, 2.30 and 1.77 for the three models, and the stack scored 79.9% on new orders.

THINK FIRST

The hold-out approach trains the first layer on only 700 orders. Why not train it on all 1,000 orders and fit the blender on its predictions for those same orders?

Answer. The first-layer models have **already seen** those orders. A full-depth forest is 100% right on its own training orders, so the blender concludes "always trust the forest". It gives the forest a weight of **9.50** and Logistic Regression **−0.08**, which means LR is ignored. On new orders the forest is right only 80% of the time, so the blender has learnt the wrong lesson. This is **data leakage**: the blender must be trained on predictions for orders the first layer did **not** see.

Blender trained on	LR	Forest	SVM
in-sample predictions (leaky)	−0.08	**9.50**	0.90
hold-out predictions	1.16	2.30	1.77

31.11 Out-of-fold predictions: no data wasted

The hold-out approach keeps 30% of the orders away from the first layer. A better way uses K-fold cross-validation. Split the orders into $K = 5$ folds. For each fold, train the first layer on the other four folds and predict the held-out fold. After five rounds, **every** order has a clean, **out-of-fold** prediction, and the blender trains on all 1,000 orders. Finally, the first layer is refitted on all the data for future predictions.

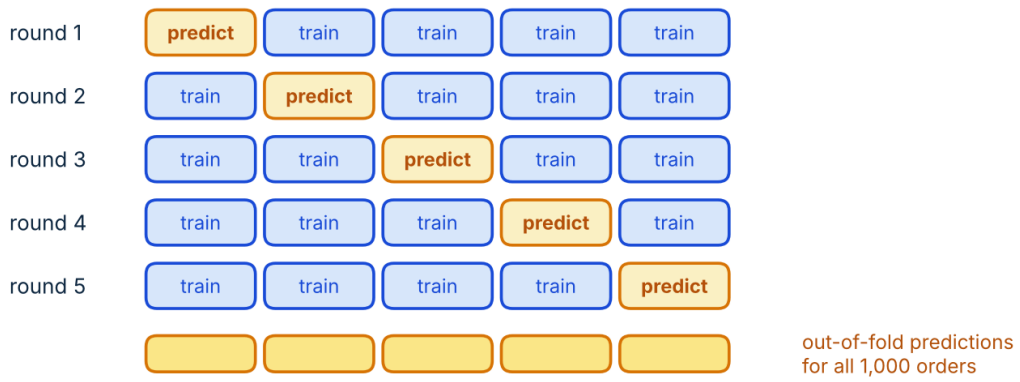


Figure 31.6. Five-fold out-of-fold predictions. In each round, the first layer is trained on four folds and predicts the fifth.

```
from sklearn.model_selection import cross_val_predict
z_rf = cross_val_predict(rnd_clf, X_train, y_train, cv=5,
                         method="predict_proba")[:, 1]
```

Deeper stacks

We can train several different blenders, for example one linear regression and one random forest, to obtain a whole **layer** of blenders, and put one more blender on top. With the hold-out approach, the training set is then split into three subsets, one per layer. To predict, a new order passes through each layer in turn. Each layer adds complexity and a risk of overfitting; in practice, one blender on top of a few diverse models is usually enough.

31.12 Stacking in scikit-learn

```
from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[("lr", log_clf), ("rf", rnd_clf), ("svc", svm_clf)],
    final_estimator=LogisticRegression(),   # the blender
    cv=5)                                   # out-of-fold predictions
stacking_clf.fit(X_train, y_train)
stacking_clf.score(X_test, y_test)          # 0.814
```

> **A correction to last year's slides**
> Last year's slides said that Scikit-Learn does not support stacking. It has provided `StackingClassifier` and `StackingRegressor` since version 0.22 (2019).

For each member, the stack uses `predict_proba()` if it exists and otherwise `decision_function()`, as with the SVM here. Setting `passthrough=True` gives the blender the original features as well. For a multi-layer stack, use another `StackingClassifier` as the `final_estimator`.

Model	Accuracy on new orders
Logistic Regression	77.1%
Random Forest	80.3%
SVM	80.2%
Soft voting	81.0%
Stacking (LR, RF, SVM)	**81.4%**
Stacking with a Gradient Boosting member added	**82.5%**

31.13 Every method on the same problem

All models below were trained on the same 1,000 COD orders and tested on the same 5,000 new orders.

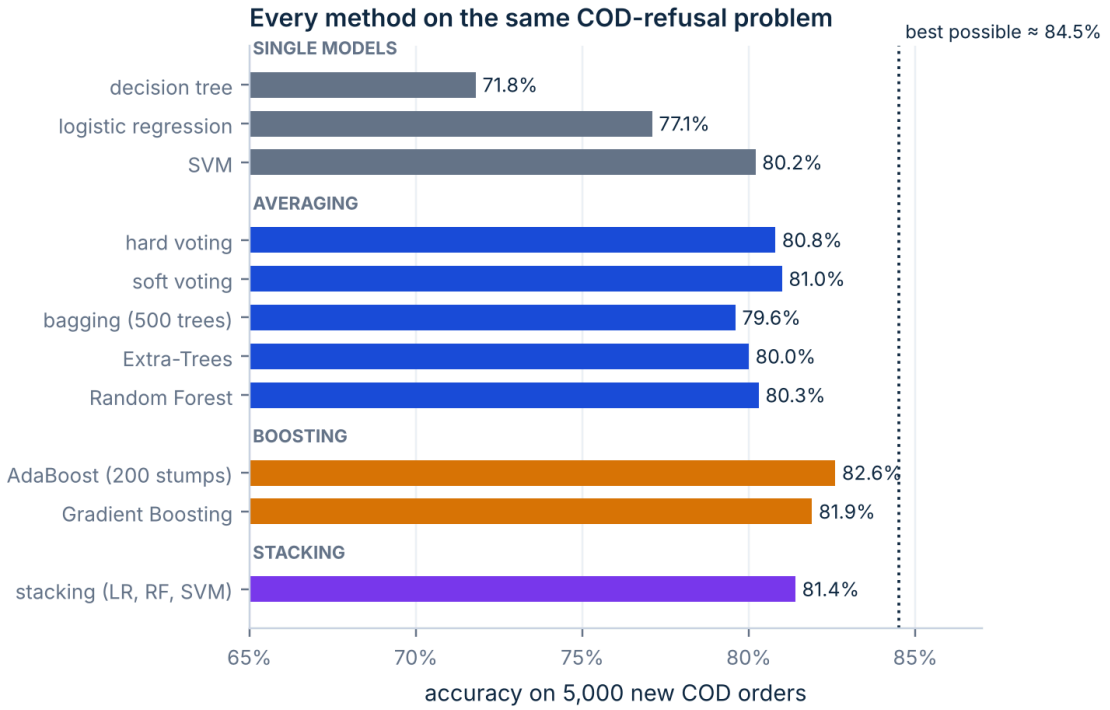


Figure 31.7. Accuracy of every method on the COD-refusal problem. The dotted line is the best accuracy any model could reach, because refusals are partly random.

- Every ensemble beats the single tree (71.8%) by a wide margin.
- Averaging methods (voting, bagging, forests) gain about eight points over a single tree.
- Boosting gains the most here, because the pattern is a sum of simple effects.
- Even a perfect model would score only about 84.5%, because refusals are partly random.

No method wins on every dataset. Use cross-validation to choose.

31.14 Which ensemble, when?

Situation	Try first	Why
A strong default for tabular data, with little tuning	**Random Forest**	Robust, parallel, OOB estimate for free.
Training must be very fast	**Extra-Trees**, **HistGradientBoosting**	Random thresholds; binned features.
Highest accuracy on tabular data, with time to tune	**Gradient Boosting** (XGBoost, LightGBM)	Reduces bias; tune `learning_rate` with early stopping.
Labels are noisy	**Random Forest** rather than AdaBoost	AdaBoost keeps boosting wrong labels.
Several good, different models already exist	**Soft voting** or **Stacking**	Combines their different errors.
You need to see which features matter	Forest + **permutation importance**	MDI alone can mislead.

Summary of Lectures 29–31

1. An ensemble combines many predictors. It works when the members are **better than chance** and **diverse**.

2. **Hard voting** counts classes; **soft voting** averages probabilities and rewards confidence.
3. **Bagging** trains one algorithm on bootstrap samples, in parallel. It keeps the bias and lowers the variance.
4. About **37%** of the orders are out of bag for each tree, which gives a free validation set.
5. **Random Forests** add random features at each node; **Extra-Trees** add random thresholds. The trees become less correlated, which lowers the variance floor $\rho\sigma^2$.
6. **AdaBoost** re-weights the hard orders; better predictors get a larger α.
7. **Gradient Boosting** fits each tree to the residuals (the negative gradients). Use shrinkage and early stopping.
8. **Stacking** trains a blender on **out-of-fold** predictions, never on predictions for orders the members have seen.

Lecture 31 · Check yourself

THINK FIRST
Your Random Forest overfits. Should you reduce `n_estimators`?

Answer. No. More trees do not cause overfitting in a forest. Regularise the trees instead (`min_samples_leaf`, `max_depth`) or reduce `max_features`.

THINK FIRST
Your Gradient Boosting model overfits. Name two remedies.

Answer. Lower the `learning_rate` (shrinkage) together with early stopping; use fewer or shallower trees; or set `subsample` below 1.

THINK FIRST
Can you train the 500 stumps of an AdaBoost ensemble on 500 machines at once?

Answer. No. Each round needs the weights produced by the previous round. Bagging and forests can be parallelised; boosting cannot (except within one tree).

THINK FIRST
Five models each reach 95% training accuracy but very different test scores. Is stacking them a good idea?

Answer. Possibly, if their errors differ. But train the blender on **out-of-fold** predictions: training accuracy says nothing about how much to trust each model.

Appendix · Key equations

	Equation	Meaning
(29.1)	$\hat{y}(x) = \text{mode}\{\, h_1(x), \dots, h_N(x) \,\}$	hard voting
(29.2)	$P(\text{majority right}) = \Sigma_{k=(n+1)/2}^{n}\, C(n,k)\, p^k\, (1-p)^{n-k}$	independent voters
(29.3)	$p \cdot \text{Tail}(n, q_A) + (1-p) \cdot \text{Tail}(n, q_B)$	voters that copy
(29.4)	$\hat{y}(x) = \arg\max_k\, (1/N)\, \Sigma_j\, \hat{p}_j(k \mid x)$	soft voting
(29.5)	$\hat{y}(x) = (1/N)\, \Sigma_j\, h_j(x)$	bagging for regression
(29.6)	$\text{Var}(\bar{h}) = \rho\sigma^2 + ((1-\rho)/N)\, \sigma^2$	variance of an average
(29.7)	$(1 - 1/m)^m \to e^{-1} \approx 0.368$	share of out-of-bag orders
(30.1)	$\text{imp}(k) \propto \Sigma\,(\, n \cdot G - n_L \cdot G_L - n_R \cdot G_R\,)$	MDI feature importance
(30.2)	$r_j = \Sigma_{\text{wrong}}\, w^{(i)} \,/\, \Sigma_i\, w^{(i)}$	AdaBoost weighted error
(30.3)	$\alpha_j = \eta\, \log((1-r_j)\,/\,r_j)$	AdaBoost predictor weight
(30.4)	$w^{(i)} \leftarrow w^{(i)}\, \exp(\alpha_j)$ if order i is misclassified	AdaBoost weight update
(30.5)	$\hat{y}(x) = \arg\max_k\, \Sigma_{j\,:\,\hat{y}j(x)\,=\,k}\, \alpha_j$	AdaBoost prediction
(30.6)	$\alpha_j = \eta\,(\, \log((1-r_j)\,/\,r_j) + \log(K-1)\,)$	SAMME, K classes
(31.1)	$-\partial L_i / \partial F(x_i) = y_i - F(x_i)$	residual = negative gradient
(31.2)	$\hat{p} = \sigma(0.944\, z_1 + 3.073\, z_2 + 0.752\, z_3 - 1.997)$	the ShopKart blender

Scikit-Learn 1.9 notes

- `SVC(probability=True)` is deprecated; use `CalibratedClassifierCV(SVC(), ensemble=False)` for soft voting.
- `AdaBoostClassifier` no longer accepts `algorithm`; `"SAMME.R"` has been removed and SAMME is always used.
- `StackingClassifier` and `StackingRegressor` have been available since version 0.22.
- The Random Forest equivalent with `BaggingClassifier` uses `DecisionTreeClassifier(max_features="sqrt")`, not `splitter="random"`.

Further reading

- A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, Chapter 7: Ensemble Learning and Random Forests.
- T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd edition, Chapters 10 (Boosting) and 15 (Random Forests).
- L. Breiman, "Bagging predictors", *Machine Learning* 24 (1996); "Random forests", *Machine Learning* 45 (2001).